Sr. No.	Topic	Sign
1	**Environmental Setup and JavaScripts Basics**	
a	Install Node.js and Visual Studio Code ,and verify installation.	
b	Create a simple Node.js file and run it using terminal.	
c	Write a Node.js script demonstrating basic JavaScript commands (console log, variables, and arithmetic operations).	
d	Create and run a simple program using variables and functions.	
e	Demonstrating use of operators and control statement in JavaScript.	
f	Write a program using loops and template literals	
2	**Core JavaScript Concept for MERN**	
a	Write a program to create a function and call them with parameters.	
b	Work with arrays and objects in JavaScript.	
c	Demonstrate the use of Arrow Function.	
d	Write a program using Array Methods. (Map, Filter, Reduces).	
e	Create a program that manipulates objects and display outputs.	
f	Implement a small JavaScript program combining functions, arrays and Objects.	
3	**React Introduction and JSX**	
a	Create a React application using Vite/CRA and display “Welcome to MERN Stack Development”	
b	Display dynamic data using JSX	
c	Write a program to declare variables and writw a function to calculate sum of 2 numbers and display results	
d	Apply Basic css styling to React Component	
e	Create a simple functional component and render in App.jsx	

f	Create a React component that display current Date and Time Dynamically using JavaScript	
4	**React Components, Props, and State**	
a	Create a multiple components and render them on a single page	
b	Pass the data between components using Props	
c	Use the useState hook to manage and update data	
d	Display the list of items stored in the state using map( )	
e	Create a form component to accept user input and display it dynamically	
f	Build a small interactive component (Eg. Counter or ToDo list)	
5	**React Routing & Data Handling**	
a	Install and configure React Router in the React application	
b	Create a multiple pages an implement and navigation between them	
c	Use Link and Routes to move between pages	
d	Fetch data from public API using Axios	
e	Display fetched data in table or list format	
f	Implement search or filter functionality on displayed data	
6	**Node.js and Express Fundamentals**	
a	Install Express and create a simple server	
b	Create Multiple Routes in Express	
c	Send HTML Content from Express Server	
d	Handle Route Parameters in Express	
e	Handle Query Parameters in Express	
f	Create a Simple JSON API using Express	

7	**Install MongoDB Community Server and MongoDB Compass**	
a	Download MongoDB Community Server	
b	Perform CRUD operations using MongoDB Compass	
c	Connect Node.js with MongoDB using Mongoose	
8	**API Creation and Integration**	
a	Create a MongoDB Scheme and model using Mongoose	
b	Implement REST API Operations	
c	Test REST APIs Using Postman	
9	**Frontend and Backend Integration using React and Axios**	
a	Create the React Frontend and connect it with Express	
b	Retrieve and Display Student Records	
c	Add, Update and Delete Student Records	
10	**Complete MERN CRUD Application**	
a	Design and Execute the Complete MERN Application	

Module-1
Practical No.1

Aim:- Environmental Setup and JavaScripts Basics.

a) Install Node.js and Visual Studio Code ,and verify installation.

Requirements:

- Node.js
- Visual Studio Code
- Command Prompt/Terminal
- Web Browser
- Basic knowledge of JavaScript

Node.js is a JavaScript runtime environment based on Chrome V8 engine.

NPM is the Node Package Manager used to manage packages.

Steps:

1. **Download**

Go to nodejs.org

Click on LTS version (it will show v24.x.x LTS)

2. **Install (Windows)**

Run the .msi file you downloaded

Next -> Next -> Accept

3. **Verify**

Open Command Prompt/Terminal and verify the Node.js installation using:

- **node –v**

Verify npm using:

- **npm –v**

4. **Install Visual Studio Code.**

It is a lightweight code editor with an integrated terminal and extensions support for MERN development.

5. Open Visual Studio Code.

b) Create a simple Node.js file and run it using terminal.

1. Create a folder named MERN_Practicals.

2. Open the folder in VS Code.

3. Create a file named app.js.

4. Add JavaScript code.

```
console.log("Welcome to Node.js");
```

console.log() – Used to print output on the terminal.

Node filename.js – Used to execute a JavaScript file.

Key Point: Every Node.js file should have a .js extension.

5. Open the terminal.

6. Run the Program using:

```
Node.js
```

7. Output.



c) Write a Node.js script demonstrating basic JavaScript commands (console log, variables, and arithmetic operations).

1. **Create a file.**
2. **Add JavaScript code.**

```
console.log("Hello,MERN Developer!");

let firstName="Vaishnavi";
let age=19;
const pi=3.14;

console.log("Fisrt Name:",firstName);
console.log("Age:",age);
console.log("Value of PI:",pi);

let num1=20;
let num2=10;

console.log("Arithmetic Operators");
console.log("Addition:",num1+num2);
console.log("Subtraction:",num1-num2);
console.log("Multiplication:",num1*num2);
console.log("Division",num1/num2);
console.log("Modulus(Reminder):",num1%num2);
console.log("Exponentiation ",num1**num2);

age=age+5;
console.log("New Age:",age);
```

3. **Open the terminal.**

4. **Run the program using:**

Node.js

5. **Output:-**

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

D:\MERN Practicals\my-app1\src>node practical1c.js
Hello,MERN Developer!
Fisrt Name: Vaishnavi
Age: 19
Value of PI: 3.14
Arithmetic Operators
Addition: 30
Subtraction: 10
Multiplication: 200
Division 2
Modulus(Reminder): 0
Exponentiation  10240000000000
New Age: 24
```

d) Create and run a simple program using variables and functions.

1. Add JavaScript code.

```
let name="Vaishnavi";

let age=19;


function display(PersonName, PersonAge)

{

   console.log("Hello "+PersonName+"!");

   console.log("You are "+ PersonAge + " Years old");

}

display(name,age);
```

2. Open the terminal.

3. Run the program using:

Node.js

4. Output:-



e) Demonstrating use of operators and control statement in JavaScript.

1. Add JavaScript code.

```
let a=20;
let b=10;

console.log("Is a is greater than b?",a>b);
console.log("Is a>15 AND b<15?",(a>15 && a<15));

if(a>b)
{
   console.log("a is greater than b");
}
else{
   console.log("b is greater than or equal to a");
}

let day=2;
switch(day){
   case 1:
      console.log("Monday");
      break;
   case 2:
      console.log("Tueday");
      break;
   case 3:
      console.log("Wednesday");
      break;
   default:
```

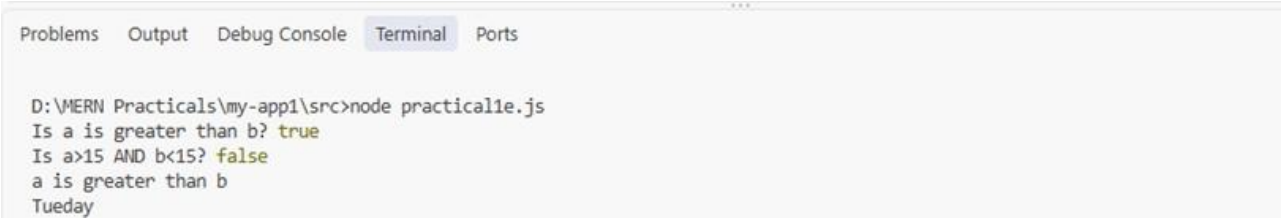
```
    console.log("Invalid Day");
}
```

2. Open the terminal.

3. Run the program using:

Node.js

5. Output:-



f) Write a program using loops and template literals.

1. Add JavaScript code.

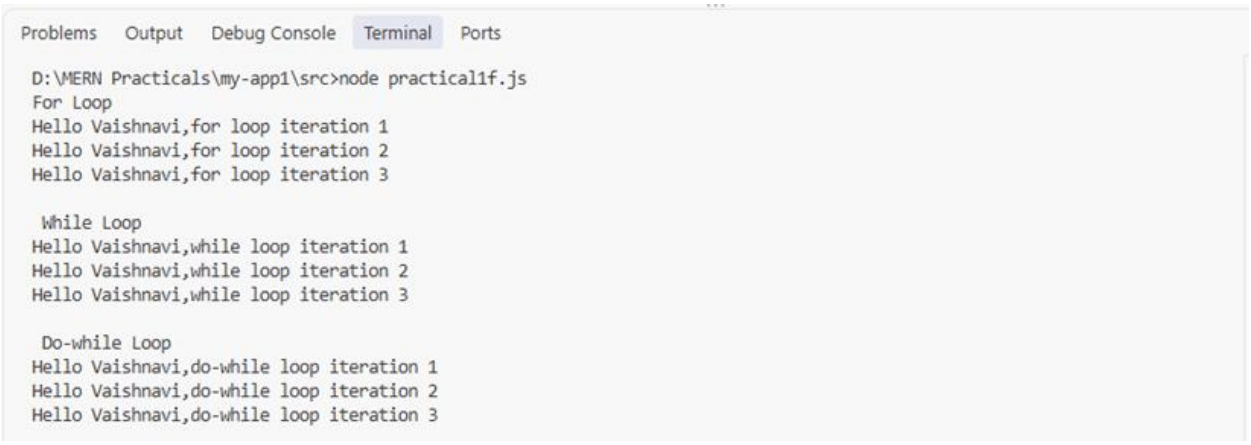
```
let name="Vaishnavi";
console.log("For Loop");
for(let i=1; i<=3; i++)
{
   console.log(`Hello ${name},for loop iteration ${i}`);
}
console.log("\n While Loop");
let j=1;
while(j<=3)
{
   console.log(`Hello ${name},while loop iteration ${j} `);
   j++;
}
console.log("\n Do-while Loop");
let k=1;
do
{
   console.log(`Hello ${name},do-while loop iteration ${k} `);
   k++;
}
while(k<=3);
```

2. Open the terminal.

3. Run the program using:

Node.js

5. Output:-



Practical No.2

Aim:- Core JavaScript Concept for MERN.

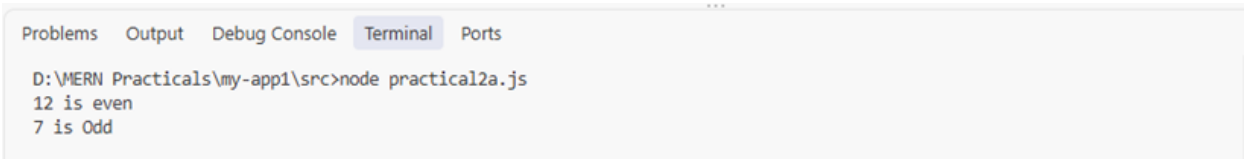
a) Write a program to create a function and call them with parameters.

1) To find whether the number is Odd or Even.

i) Add JavaScript Code.

```
function checkEvenOdd(num)
{
   if (num%2==0)
   {
      console.log(num+" is even");
   }
   else
   {
      console.log(num+" is Odd");
   }
}
checkEvenOdd(12);
checkEvenOdd(7);
```

ii) Output:-

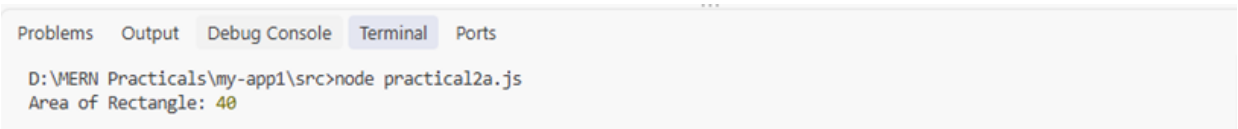


2) To find the Area of Rectangle.

i) Add JavaScript Code.

```
function rectangleArea(length,width)
{
   return length*width;
}
let area=rectangleArea(8,5);
console.log("Area of Rectangle:",area);
```

ii) Output:-



3) To display the Student Details.

i) Add JavaScript Code.

```
function stud(name,age,course)
{
   console.log("Name:",name);
   console.log("Age:",age);
   console.log("Course:",course);
}
stud("Vaishnavi",19,"MERN Stack");
```

ii) Output:-



b) Work with arrays and objects in JavaScript.

1. Open Visual Studio Code.

2. Create a new file.

3. Add JavaScript Code.

```
let fruits=["Apple","Banana","Mango","Orange"];

let student={
   name:"Vaishnavi",
   age:19,
   course:"BSc IT"
};

console.log("First Fruit:",fruits[0]);

console.log("Second Fruit:",fruits[1]);


console.log("Student Name:",student.name);

console.log("Course:",student.course);
```

4.Output:-



c) Demonstrate the use of Arrow Function.

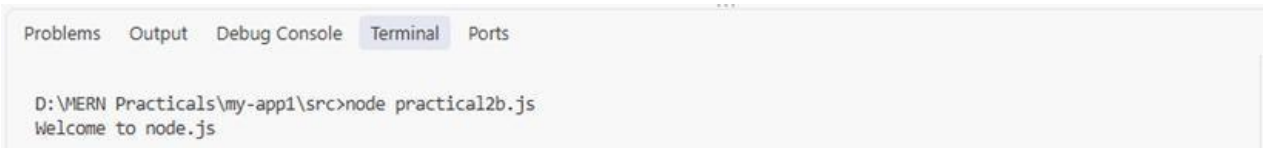
Definition:- Arrow function is shorter way to write a function.

1) Without Parameter.

i) Add JavaScript Code.

```
const display=()=>
{
   console.log("Welcome to node.js");
};
display();
```

ii) Output:-

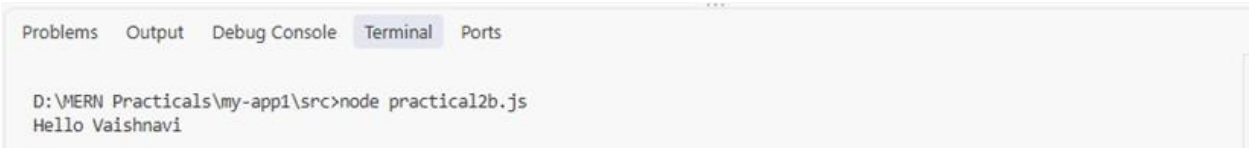


2) One Parameter.

i) Add JavaScript Code.

```
const display=(name)=>
{
   console.log("Hello",name);
};
display("Vaishnavi");
```

ii) Output:-

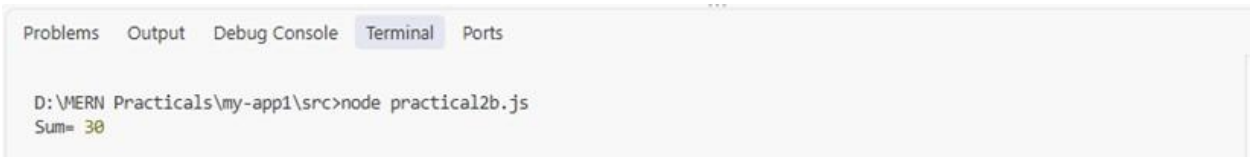


3) Arrow functions with multiple parameters.

i) Add JavaScript Code.

```
const add=(a,b)=>
{
   return a+b;
};
console.log("Sum=",add(10,20));
```

ii) Output:-

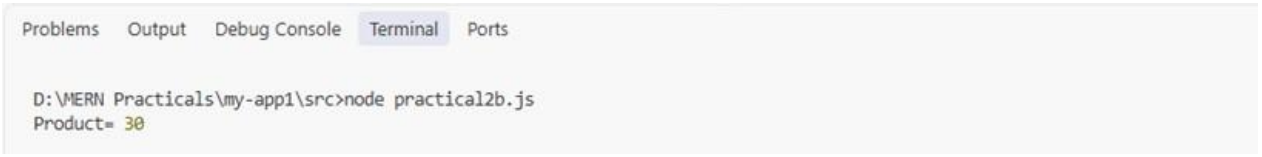


4) Arrow functions with implicit return.

i) Add JavaScript Code.

```
const multiply=(a,b)=>a*b;
console.log("Product=",multiply(5,6));
```

ii) Output:-

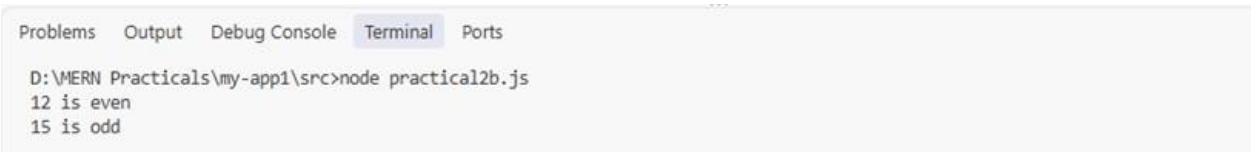


5) Arrow function to check whether the number is Even or Odd.

i) Add JavaScript Code.

```
const checkSum=(num)=>{
    if (num%2==0){
        console.log(num+" is even");
    }
    else{
        console.log(num+" is odd");
    }
}
checkSum(12);
checkSum(15);
```

ii) Output:-



e) Write a program using Array Methods. (Map, Filter, Reduces).

Map :- Map methods creates new array by applying a function to every element of original array.

Filter :- It creates a new array containing only the elements that satisfy a condition.

Reduces:- Reduces all array elements to Single Value.

1. Add JavaScript Code.

```
const numbers=[10,15,20,25,30];

const doubled=numbers.map(num=>num*2);

console.log("Doubled Array:",doubled);

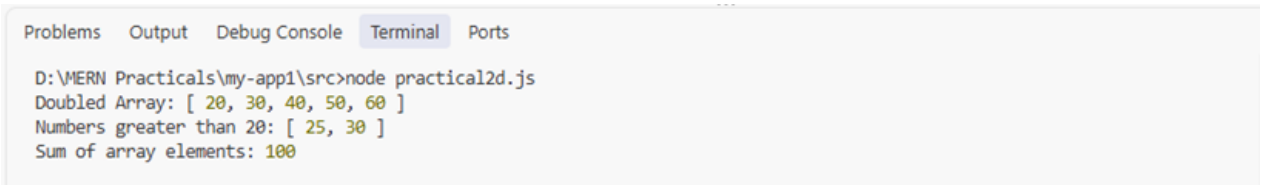
const greaterthan20=numbers.filter(num=>num>20);

console.log("Numbers greater than 20:",greaterthan20);

const sum=numbers.reduce((total,num)=>total+num,0);

console.log("Sum of array elements:",sum);
```

2. Output:-



e) Create a program that manipulates objects and display outputs.

1. Add JavaScript Code.

```
let Student={name:"Vaishnavi",age:19,course:"BSc IT",marks:95};

console.log("Original Student Details:");

console.log(Student);

Student.city="Bhiwandi";

Student.marks=96;

delete Student.age;

console.log("\n Updated Student Details");

console.log(Student);

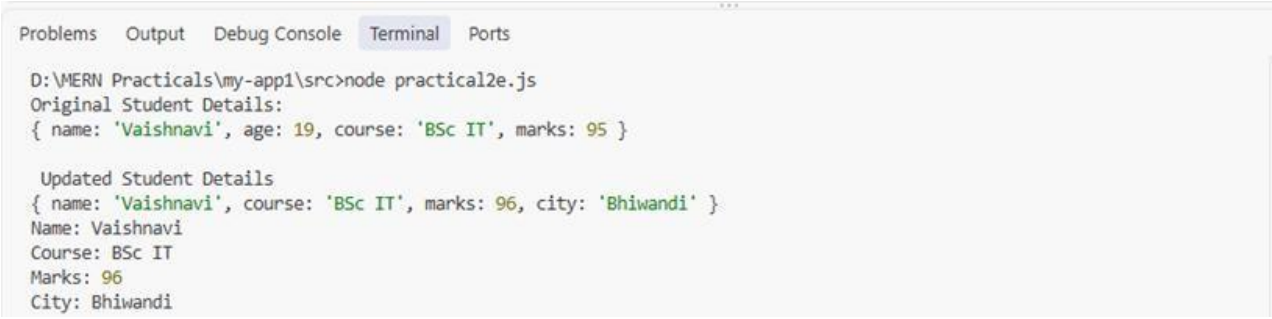
console.log("Name:",Student.name);

console.log("Course:",Student.course);

console.log("Marks:",Student.marks);

console.log("City:",Student.city);
```

2.Output:-



f) Implement a small JavaScript program combining functions, arrays and Objects.

1. Add JavaScript Code.

```
const employees = [
    {
        empID: 101,
        name: "Vaishnavi",
        department: "IT",
        salary: 50000
    },
    {
        empID: 102,
        name: "Neha",
        department: "HR",
        salary: 45000
    },
    {
        empID: 103,
        name: "Gita",
        department: "Finance",
        salary: 40000
    }
];
function displayEmployee(employeeList) {
    console.log("--- Employee Details ---");
```

```
    employeeList.forEach(function(employee) {
            console.log("Employee ID:", employee.empID);
        console.log("Name:", employee.name);
        console.log("Department:", employee.department);
        console.log("Salary:", employee.salary);
        console.log(" ---------- ");
         });
}
function calculateTotalSalary(employeeList) {
        let totalSalary = 0;
        employeeList.forEach(function(employee) {
                totalSalary += employee.salary;
        });
        console.log("Total Salary =", totalSalary);
}
displayEmployee(employees);
calculateTotalSalary(employees);
```

2.Output:-


Problems
Output
Debug Console
Terminal
Ports
D:\MERN Practicals\my-app1\src>node practical2f.js
--- Employee Details ---
Employee ID: 101
Name: Vaishnavi
Department: IT
Salary: 50000
Employee ID: 102
Name: Neha
Department: HR
Salary: 45000
Employee ID: 103
Name: Gita
Department: Finance
Salary: 40000
Total Salary = 135000

Important Methods / Functions

1. function

Used to create a reusable block of JavaScript code.

2. console.log( )

Used to display output in the console.

3. map( )

Creates a mew array by applying a function to every element.

4. filter( )

Creates a new array containing elements that satisfy a condition.

5. reduce( )

Reduces an array to a single value.

6. Arrow Function =>

Provides a shorter syntax for writing functions.

7. Object Property Access

Object.property

Used to access a property of an object.

8. return

Returns a value from a function.

Practical No.3

Aim:- React Introduction and JSX.

a) Create a React application using Vite/CRA and display "Welcome to MERN Stack Development".

Requirements:

- Visual Studio Code
- Node.js and npm
- React
- Vite
- Web Browser

1. Open Command Prompt / VS Code Terminal

2. Run command:

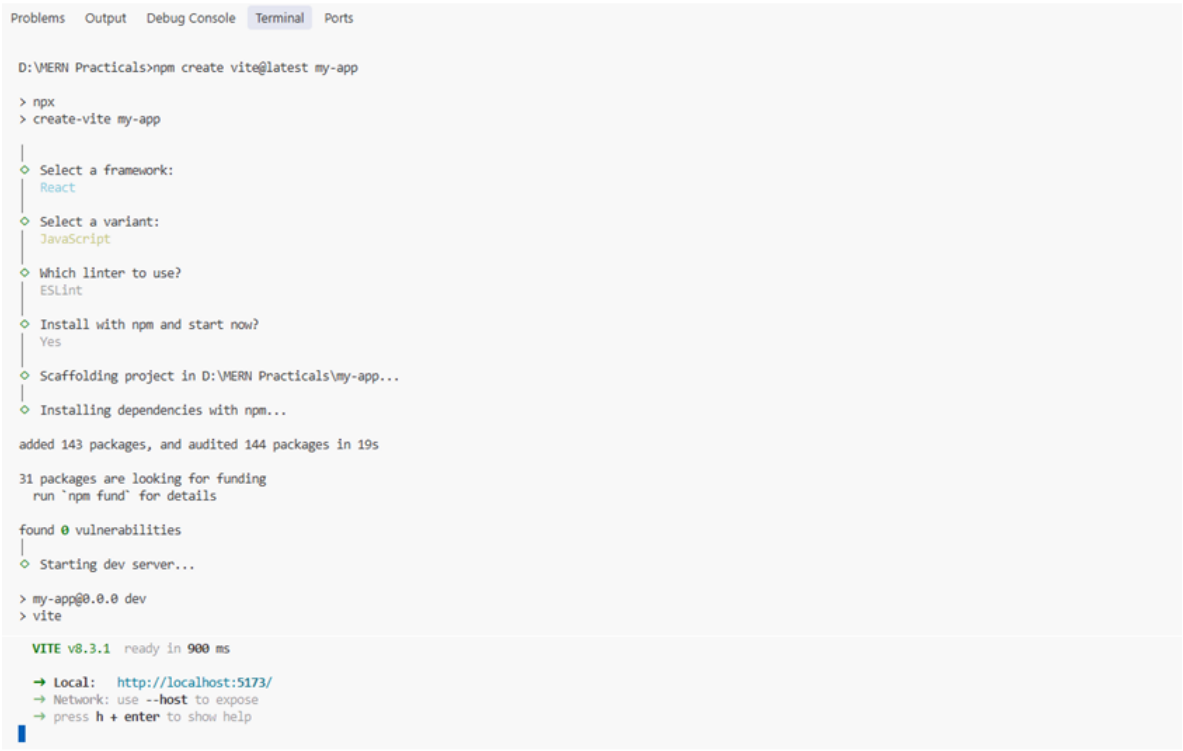
npm create vite@latest my-app

Select Framework – react

Select Variant – JavaScript

Which linear to use? – ESLint

Install with npm and start now? – Yes



3.

Open this folder in VS Code

4. Go to src folder -> Open App.jsx

Delete all existing code and write this:

```
import React from "react";
import "./App.css";
function App()
{
  return(
    <div className="container">
      <h1>Welcome to MERN Stack Development</h1>
    </div>
  )
}
export default App;
```

5. Open App.css

Delete all existing code and write this:

```
.container{
  text-align: center;
  margin-top: 50 px;
}
h1{
  color: red;
}
```

6. Run in the Terminal:

npm run dev

Open link http://localhost:5174 in Browser

7. Output



b) Display dynamic data using JSX.

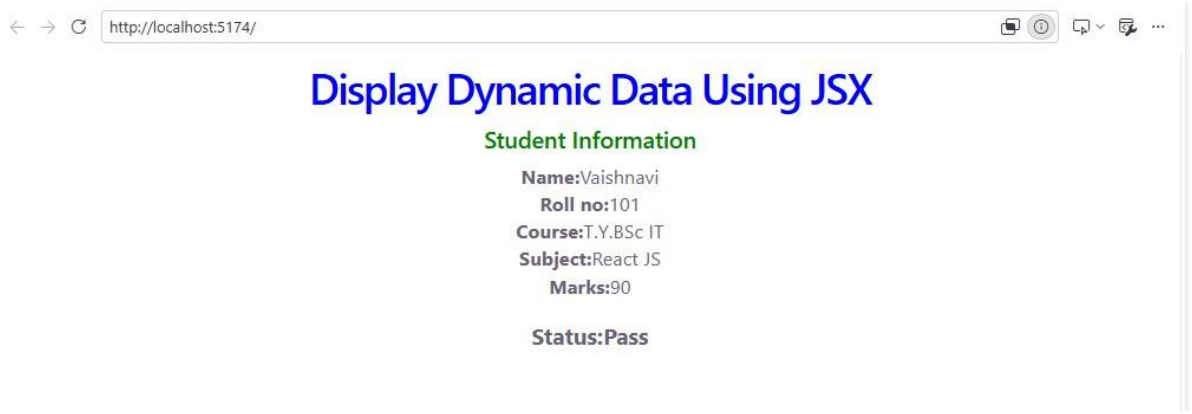
In src/App.jsx

```
import React from "react";
import "./App.css";
function App()
{
  const student_name="Vaishnavi";
  const rollno=101;
  const course="T.Y.BSc IT";
  const subject="React JS";
  const marks=90;
  return(
    <div className="container">
      <h1>Display Dynamic Data Using JSX</h1>
      <h2>Student Information</h2>
      <p><b>Name:</b>{student_name}</p>
      <p><b>Roll no:</b>{rollno}</p>
      <p><b>Course:</b>{course}</p>
      <p><b>Subject:</b>{subject}</p>
      <p><b>Marks:</b>{marks}</p>
      <h3>Status:{marks>=35? "Pass":"Fail"}</h3>
    </div>
  );
}
export default App;
```

In src/App.css

```
.container{
  width: 450 px;
  margin: 50 px auto;
  background:  white;
  padding: 20 px;
  border-radius: 10 px;
  box-shadow: 0 0 10 px gray;
}
h1{
  color: blue;
  text-align: center;
}
h2{
  color: green;
}
```

Output:



c) Write a program to declare variables and writw a function to calculate sum of 2 numbers and display results.

In src/App.jsx

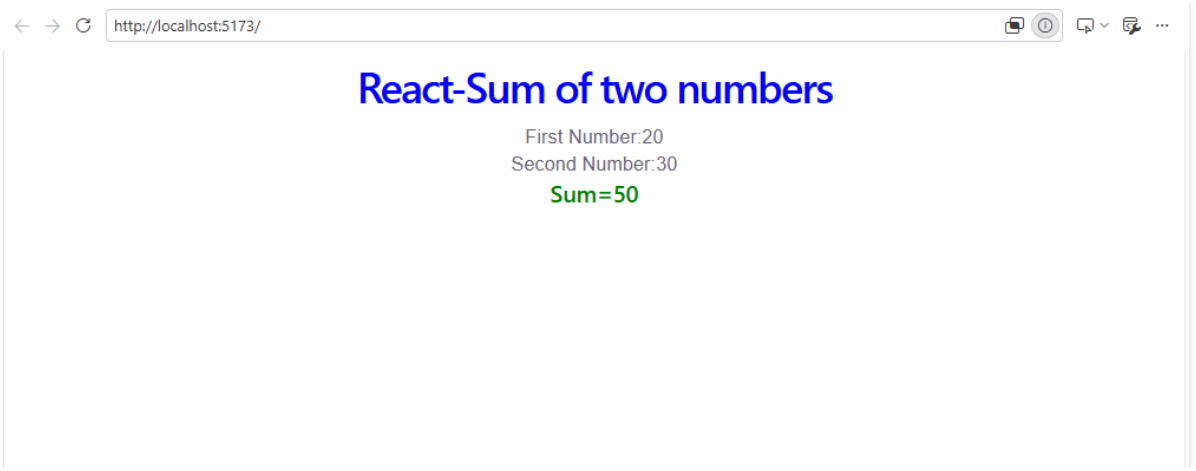
```
import React from "react";
import "./App.css";
function App()
{
  const num1=20;
  const num2=30;
  function calculateSum(a,b){
    return a+b;
  }
  return(
    <div>
      <h1>React-Sum of two numbers</h1>
      <p>First Number:{num1}</p>
      <p>Second Number:{num2}</p>
      <h2>Sum={calculateSum(num1,num2)}</h2>
    </div>
  );
}
export default App;
```

In src/App.css

```
div{
  text-align: center;
  margin-top: 50 px;
  font-family: Arial, Helvetica, sans-serif;
```

```
}
h1{
  color: blue;
}
h2{
  color: green;
}
```

Output:



d) Apply Basic css styling to React Component.

In src/App.jsx

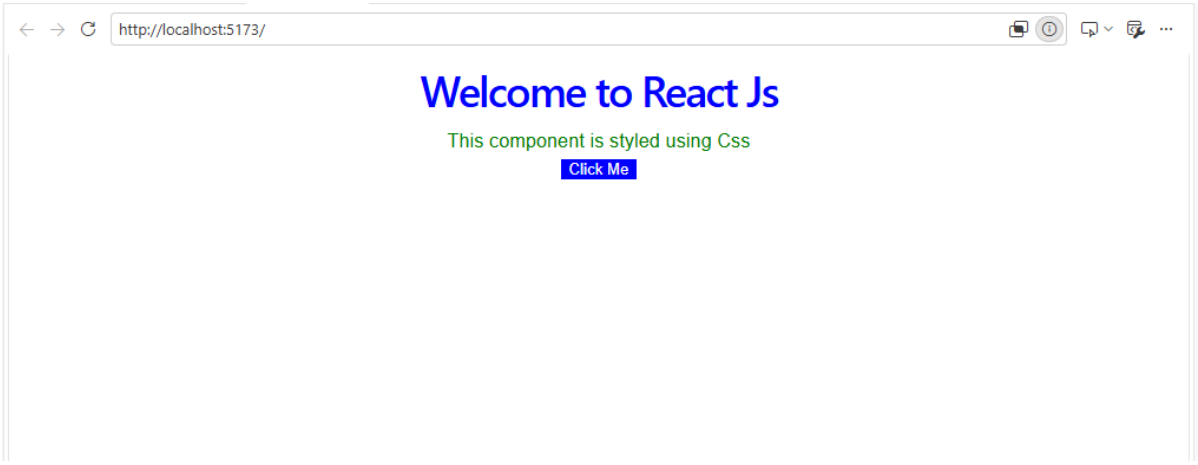
```
import React from "react";
import "./App.css";
function App()
{
  return(
    <div className="container">
      <h1>Welcome to React Js</h1>
      <p>This component is styled using Css</p>
      <button>Click Me</button>
    </div>
  );
}
export default App;
```

In src/App.css

```
.container{
  text-align: center;
  margin-top: 50 px;
  font-family: Arial, Helvetica, sans-serif;
}
h1{
  color: blue;
}
```

```
p{
  color: green;
}
button{
  background-color: blue;
  color: white;
  padding: 10 px, 20 px;
  border: none;
  cursor: pointer;
}
button:hover{
  background-color: blue;
}
```

Output:



e) Create a simple functional component and render in App.jsx.

1. Create Page Components

Inside the Src folder, create a folder

Inside it, create:

Student.jsx

In src/Student.jsx

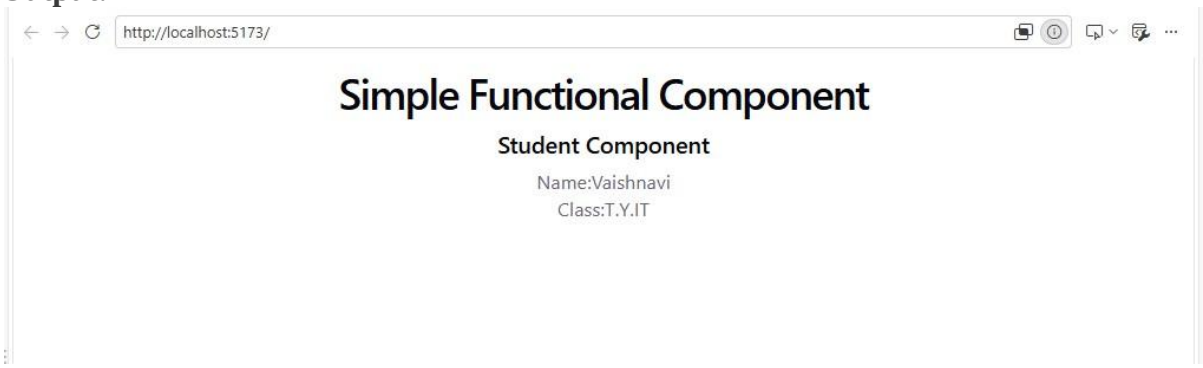
```
import React from "react";
function Student(){
   return(
      <div>
         <h2>Student Component</h2>
         <p>Name:Vaishnavi</p>
         <p>Class:T.Y.IT</p>
      </div>
   );
}
export default Student;
```

In src/App.jsx

```
import React from "react";
import Student from "./Student";
function App()
{
  return(
    <div className="container">
      <h1>Simple Functional Component</h1>
      <Student/>
```

```
    </div>
  );
}
export default App;
```

Output:



f) Create a React component that display current Date and Time Dynamically using JavaScript.

1. Create Page Components

Inside the Src folder, create a folder

Inside it, create:

DateTime.jsx

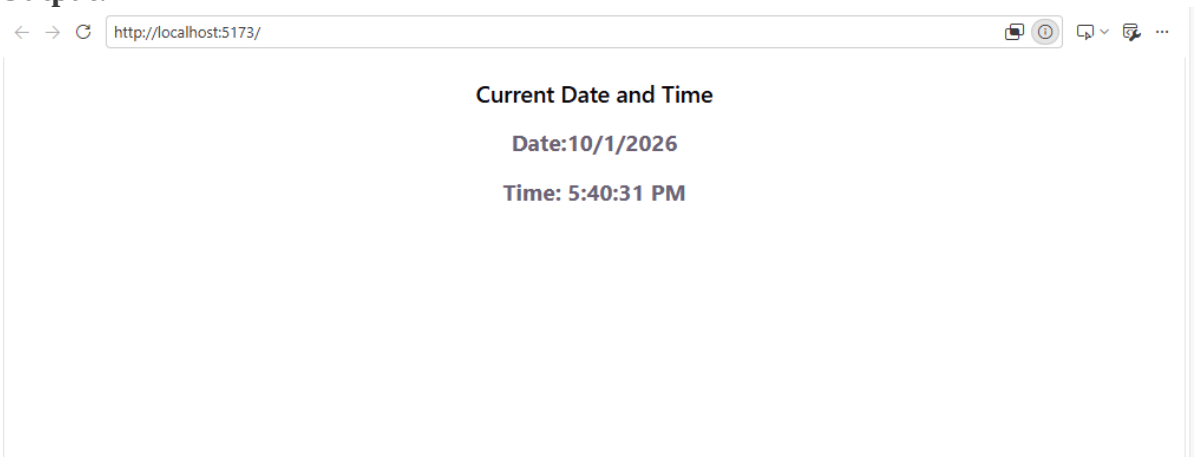
In src/ DateTime.jsx

```
import React, {useState, useEffect} from "react";
function Datetime(){
const [datetime, setDatetime] = useState(new Date());
useEffect(() => {
   const timer = setInterval(() => {
      setDatetime(new Date());
   }, 1000);
   return () => clearInterval(timer);
}, []);
return (
   <div style={{ textAlign: "center", marginTop: "20px" }}>
      <h2>Current Date and Time</h2>
      <h3>Date:{datetime.toLocaleDateString()}</h3>
      <h3>Time: {datetime.toLocaleTimeString()}</h3>
   </div>
);
}
export default Datetime;
```

In src/ App.jsx

```
import React from "react";
import Datetime from "./DateTime";
function App(){
  return(
    <div>
      <Datetime/>
    </div>
  );
}
export default App;
```

Output:-



Important Methods / Functions.

1. function App( )

Used to Create a reusable Functional Component in React.

2. return( )

Used to return JSX, which is the HTML-like code to be displayed on screen.

3. export default

Used to export a component so it can be imported and used in other files.

4. JSX Expression { }

Used to embed and display JavaScript variables and expressions inside JSX.

5. import

Used to import CSS files and other components into the current file.

6. style = { { } }

Used to apply inline CSS styling to JSX elements in React.

7. className

Used to apply CSS classes from an external CSS file, instead of using ‘class’.

8. function call – calculateSum( )

Used to create a reusable block of code and call it inside JSX to get a result.

9. new Date ( )

Used to create an object that contains the current date and time.

10. toLocaleDateString( )

Used to display the current date in a readable string format.

11. toLocaleTimeString( )

Used to display the current time in a readable string format.

Practical No.4

Aim:- React Components, Props, and State.

a) Create a multiple components and render them on a single page.

Requirements:

- VS Code
- Node.js
- React + Vite

1. Create a new React project using

npm create vite@latest practical-4 ----- template react

2. Go inside folder:

cd practical-4

3. install dependencies: npm install

In src/Header.jsx

```
function Header(){
   return(
      <div>
         <h1>MERN Practical</h1>
         <hr/>
      </div>
   );
}
export default Header;
```

In src/Student.jsx

```
function Student(props){
   return(
      <div>
         <h3>Student Information</h3>
         <p>Name:{props.name}</p>
         <p>Class:{props.className}</p>
```

```
            <p>Roll No:{props.rollno}</p>
            <hr/>
        </div>
    );
}
export default Student;
```

In src/Footer.jsx

```
function Footer(){
    return(
        <div>
            <h4>BNN College, Bhiwandi</h4>
            <p>React Practical</p>
        </div>
    );
}
export default Footer;
```

In src/Counter.jsx

```
import { useState } from "react";
function Counter(){
    const [count,setCount]=useState(0);
    return(
        <div>
            <h3>Counter using State</h3>
```

```
            <h2>{count}</h2>
            <button onClick={()=>setCount(count+1)}></button>
            <button onClick={()=>setCount(count-1)}></button>
            <hr/>
        </div>
    );
}
export default Counter;
```

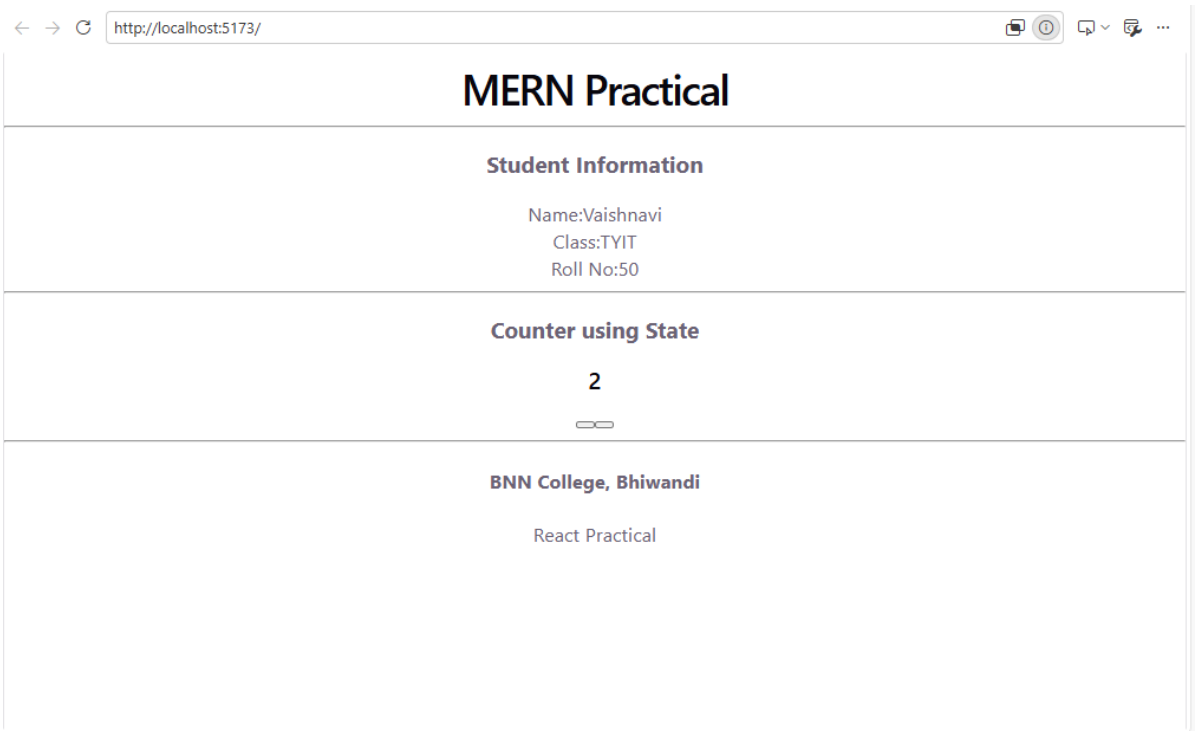
In src/App.jsx

```
import Header from "./Header";
import Student from "./Student";
import Counter from "./Counter";
import Footer from "./Footer";
function App(){
  return(
    <div style={{textAlign:"center"}}>
      <Header/>
      <Student name="Vaishnavi" className="TYIT" rollno="50"/>
      <Counter/>
      <Footer/>
    </div>
  );
```

```
}

export default App;
```

Output:-



b) Pass the data between components using Props.

In src/App.jsx

```
import React from "react";
import Employee from "./Employee";
function App(){
  return(
    <div>
      <Employee name="Vaishnavi" department="IT" salary={50000}/>
    </div>
  );
}
export default App;
```

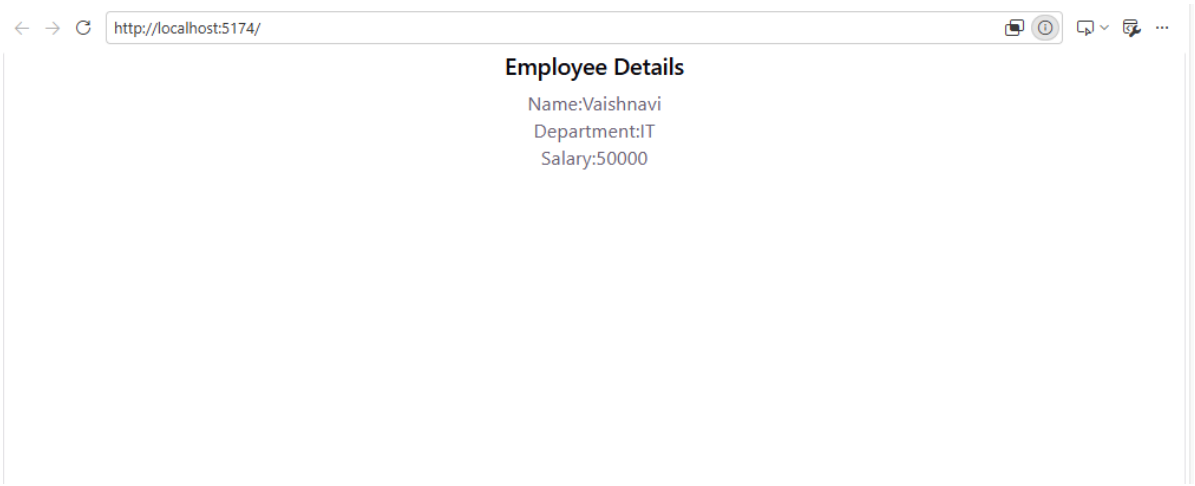
In src/Employee.jsx

```
function Employee(props){
    return(
        <div>
            <h2>Employee Details</h2>
            <p>Name:{props.name}</p>
            <p>Department:{props.department}</p>
            <p>Salary:{props.salary}</p>
        </div>
    );
```

```
}

export default Employee;
```

Output:-



c) Use the useState hook to manage and update data.

(Hook :- It is a special function that allows you to use react features such as state and lifecycle)

useState:- Built-in react features that lets you to add local state to functional components.

It returns and array with the current state value and setter function to update that value and trigger a component re-render.

In src/App.jsx

```
import React, { useState } from "react";

function App(){

const[name,setName]=useState("Vaishnavi");

const updateName=()=>{

   setName("Hello");

 };

 return(

   <div>

     <h2>useState hook Example</h2>

     <h3>{name}</h3>

     <button onClick={updateName}>Update data</button>

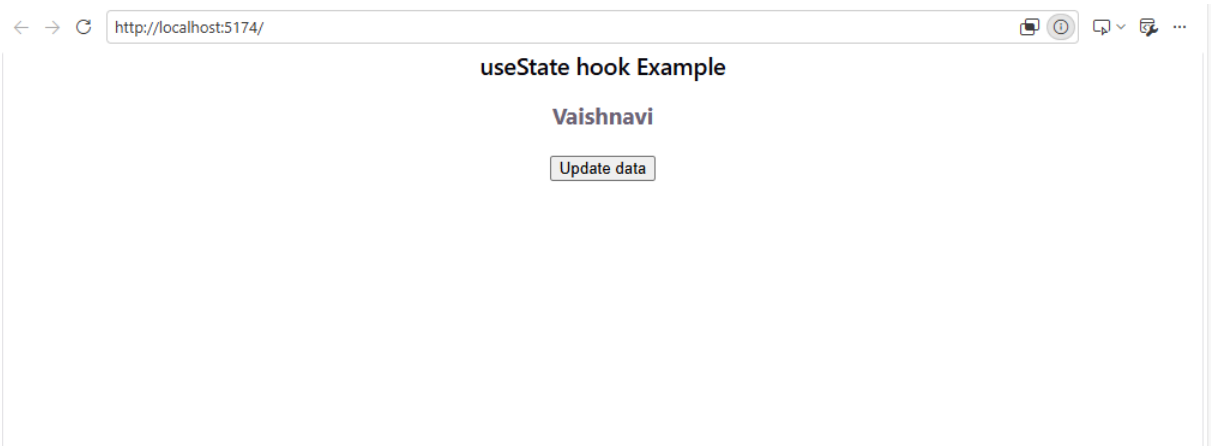
   </div>

 );
```

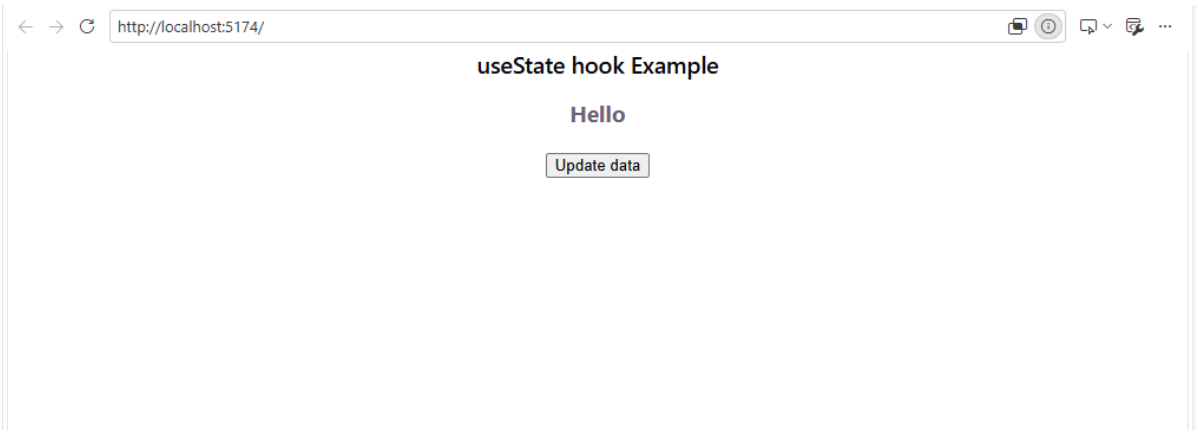
```
}

export default App;
```

Output:-



When we click Update data:



d) Display the list of items stored in the state using map( ).

map( ):- Map function is used to Dynamically loop through an array and render a list of JSX elements.

In src/App.jsx

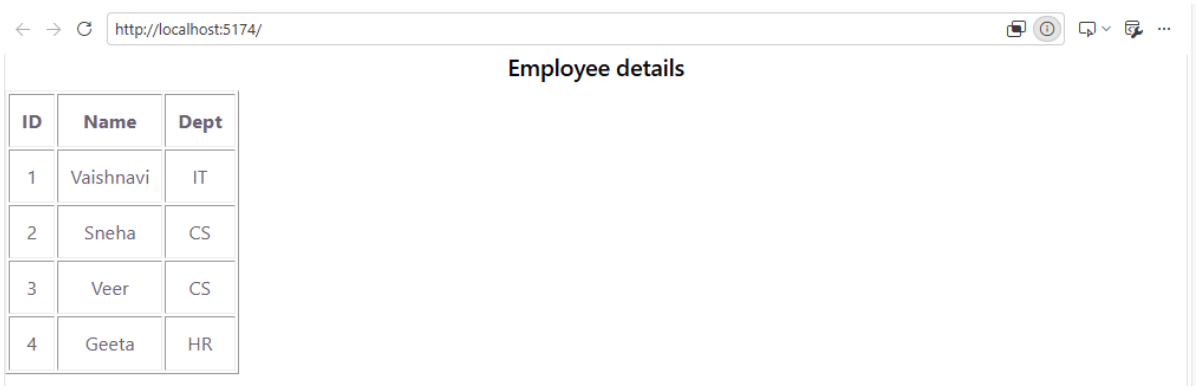
```
import React from "react";
function App() {
  const employees = [
    {
      id: 1,
      name: "Vaishnavi",
      dept: "IT",
    },
    {
      id: 2,
      name: "Sneha",
      dept: "CS",
    },
    {
      id: 3,
      name: "Veer",
      dept: "CS",
    },
    {
      id: 4,
      name: "Geeta",
      dept: "HR",
    }
  ];
  return (
    <div>
```

```
        <h2>Employee details</h2>
        <table border="1" cellPadding="10">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Dept</th>
            </tr>
          </thead>

          <tbody>
            {
              employees.map((employee) => (
                <tr key={employee.id}>
                  <td>{employee.id}</td>
                  <td>{employee.name}</td>
                  <td>{employee.dept}</td>
                </tr>
              ))
            }
          </tbody>
        </table>
      </div>
    );
}

export default App;
```

Output:-


http://localhost:5174/
Employee details
ID
Name
Dept
1
Vaishnavi
IT
2
Sneha
CS
3
Veer
CS
4
Geeta
HR

e) Create a form component to accept user input and display it dynamically.

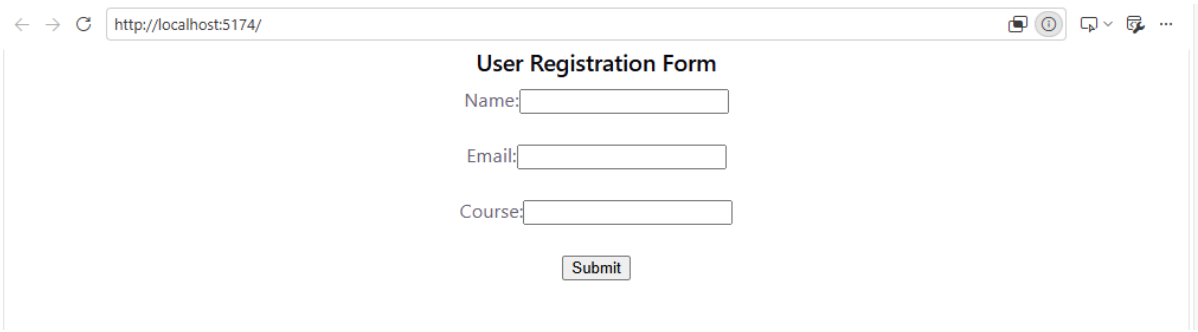
In src/App.jsx

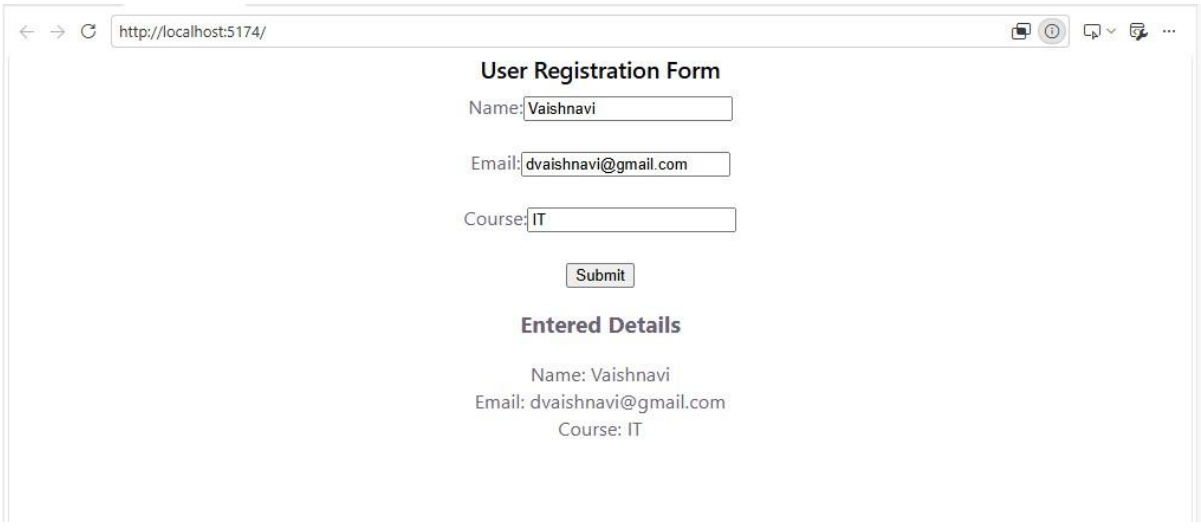
```
import { useState } from "react";
function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [course, setCourse] = useState("");
  const [submitted, setSubmitted] = useState(false);
  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(true);
  };
  return (
    <div>
      <h2>User Registration Form</h2>
      <form onSubmit={handleSubmit}>
        <label>Name:</label>
        <input
          type="text"
          value={name}
          onChange={(e) =>
            setName(e.target.value)
          }
        />
        <br /><br />
```

```
        <label>Email:</label>
        <input
          type="email"
          value={email}
          onChange={(e) =>
            setEmail(e.target.value)
          }
        />
        <br /><br />
        <label>Course:</label>
        <input
          type="text"
          value={course}
          onChange={(e) =>
            setCourse(e.target.value)
          }
        />
        <br /><br />
        <button type="submit">Submit</button>
      </form>
      {submitted && (
        <div>
          <h3>Entered Details</h3>
          <p>Name: {name}</p>
          <p>Email: {email}</p>
```

```
            <p>Course: {course}</p>
          </div>
        )}
      </div>
    );
}
export default App;
```

Output:-





f) Build a small interactive component (Eg., Counter or ToDo list).

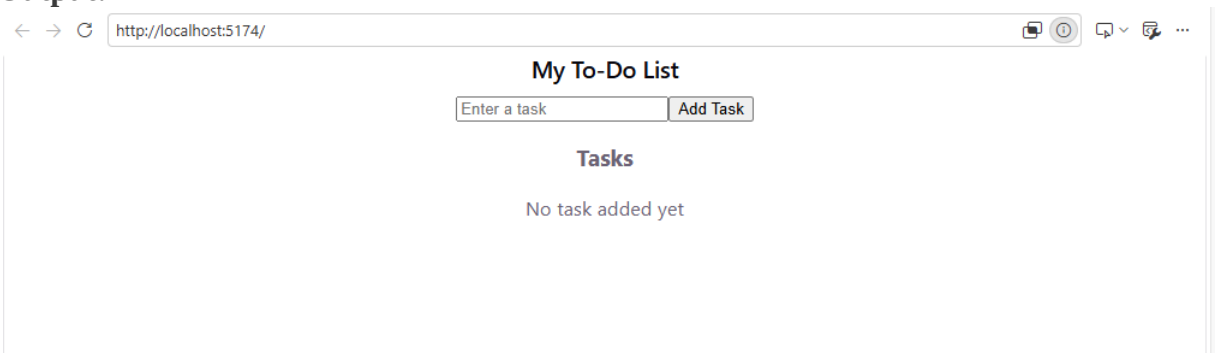
In src/App.jsx

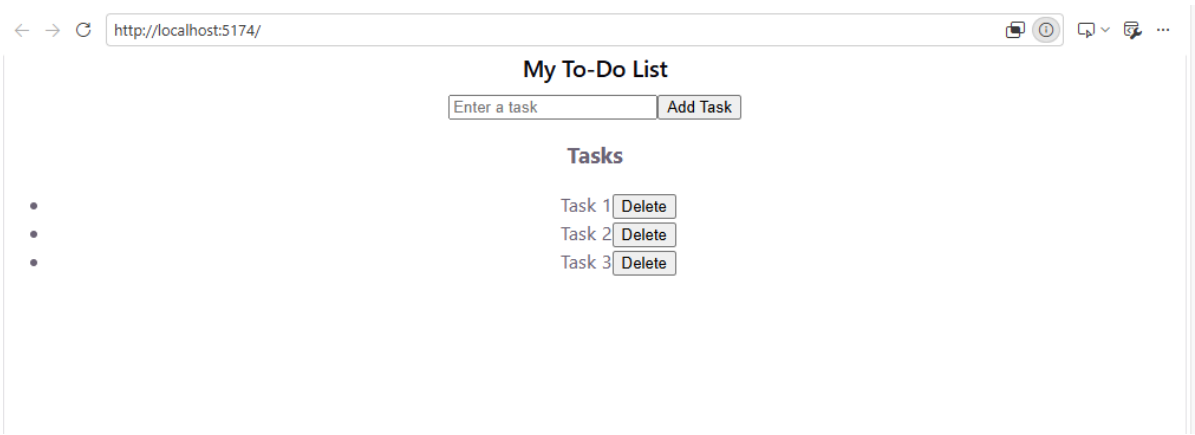
```
import { useState } from "react";
function App() {
  const [task, setTask] = useState("");
  const [tasks, setTasks] = useState([]);
  const addTask = () => {
    if (task.trim() === "") {
      alert("Please enter a task");
      return;
    }
    setTasks([
      ...tasks,
      {
        id: Date.now(),
        name: task,
        completed: false
      }
    ]);
    setTask("");
  };
  const deleteTask = (id) => {
    setTasks(tasks.filter((item) => item.id !== id));
  };
  const completeTask = (id) => {
    setTasks(
      tasks.map((item) =>
        item.id === id
          ? { ...item, completed: !item.completed }
          : item
      )
```

```
    );
  };
  return (
    <div>
      <h2>My To-Do List</h2>
      <input
        type="text"
        placeholder="Enter a task"
        value={task}
        onChange={(e) => setTask(e.target.value)}
      />
      <button onClick={addTask}>Add Task</button>
      <h3>Tasks</h3>
      {tasks.length === 0 ? (
        <p>No task added yet</p>
      ) : (
        <ul>
          {tasks.map((item) => (
            <li key={item.id}>
              <span
                onClick={() => completeTask(item.id)}
                style={{
                  textDecoration: item.completed
                    ? "line-through"
                    : "none",
                  cursor: "pointer"
                }}
              >
                {item.name}
              </span>
              <button
```

```
                    onClick={() => deleteTask(item.id)}
                  >
                    Delete
                  </button>
                </li>
              ))}
            </ul>
          )}
        </div>
    );
}
export default App;
```

Output:-





Important Methods / Functions

1. `function App()`
Used to create a reusable Functional Component in React.

2. `return()`
Used to return JSX, which is the HTML-like code to be displayed on screen.

3. `useState()`
Used to create and manage state in a functional component. It returns a value and a function to update it.

4. `props`
Used to pass data from parent component to child component.

5. `setCount()` / `setName()`
Used to update the state value and re-render the component with new data.

6. `onClick={ }`
Used to handle click events on buttons and other elements in React.

7. `onChange={ }`
Used to handle input field changes and accept user input dynamically.

8. `map()`
Used to display a list of items stored in state or array by iterating over them.

9. `export default`
Used to export a component so it can be imported and used in other files.

Practical No.5

Aim:- React Routing & Data Handling.

a) Install and configure React Router in the React application.

Requirements:

- VS Code
- Node.js
- React application created using vite

Create a React Application

Steps

1. Open VS Code

2. Open Terminal → New Terminal.

3. Type this commands in Terminal.

i. npm create vite@latest react-routing
ii. npm intall react-router-dom@6

In src/App.jsx

```
import React from "react";
import {
  BrowserRouter as Router,
  Routes,
  Route,
  Link
} from "react-router-dom";
function Home() {
  return (
    <div>
      <h2>Home Page</h2>
    </div>
```

```
  );
}
function About() {
  return (
    <div>
      <h2>About Page</h2>
      <ul>
        <li>
          <Link to="/team">Our Team</Link>
        </li>
        <li>
          <Link to="/company">Our Company</Link>
        </li>
      </ul>
    </div>
  );
}
function Team() {
  return <h2>Team Page</h2>;
}
function Company() {
  return <h2>Company Page</h2>;
}
function Contact() {
  return <h2>Contact Page</h2>;
}
function App() {
  return (
```

```
    <Router>
      <nav>
        <ul>
          <li>
            <Link to="/">Home</Link>
          </li>
          <li>
            <Link to="/about">About</Link>
          </li>
          <li>
            <Link to="/contact">Contact</Link>
          </li>
        </ul>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/team" element={<Team />} />
        <Route path="/company" element={<Company />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </Router>
  );
}
export default App;
```

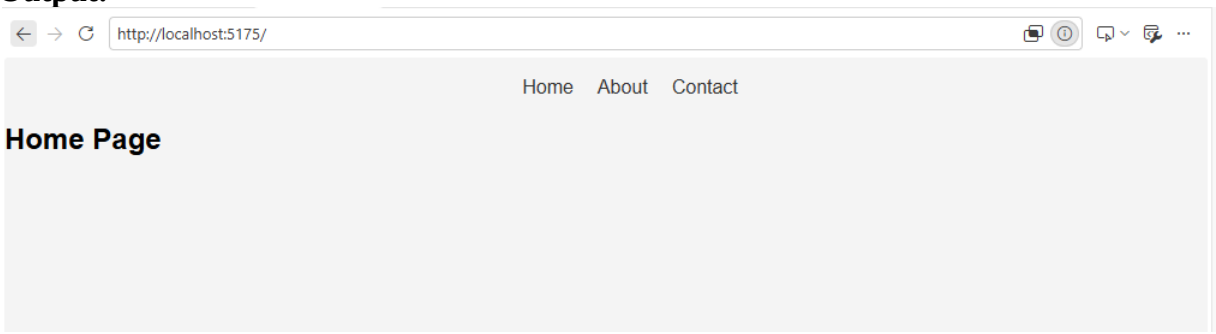
In src/index.css

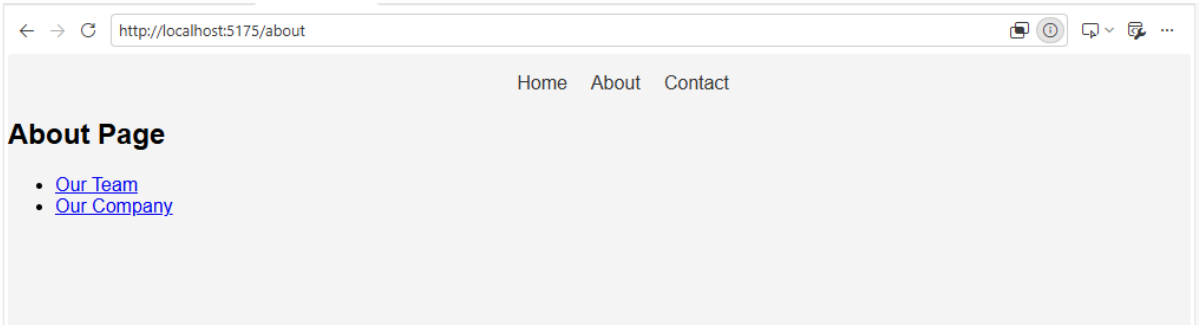
```
body {
    font-family: Arial, sans-serif;
```

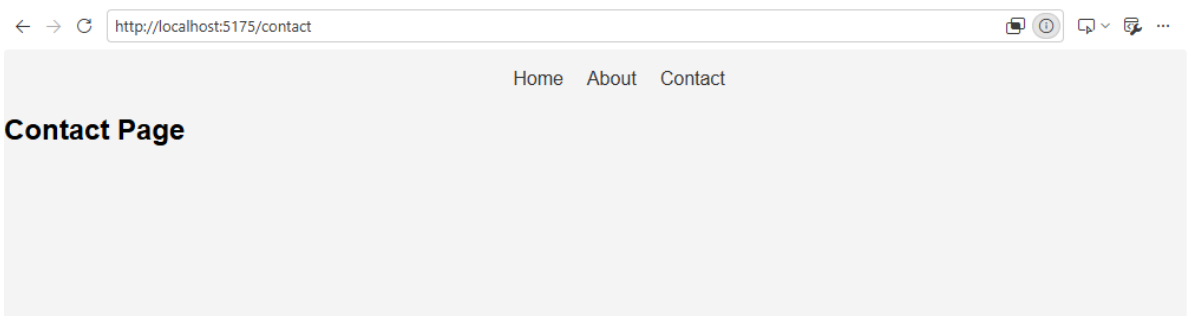
```
    background-color: #f4f4f4;
    margin: 0;
    padding: 0;
}
h1 {
    text-align: center;
    color: red;
}
nav ul {
    display: flex;
    justify-content: center;
    list-style: none;
}
nav li {
    margin: 0 10px;
}
nav a {
    text-decoration: none;
    color: #333;
}
button {
    display: block;
    margin: 20px auto;
    padding: 10px 20px;
    background-color: #007bff;
    color: white;
    border: none;
    cursor: pointer;
```

```
}
button:hover {
    background-color: blue;
}
```

Output:-







b) Create a multiple pages an implement and navigation between them.

In src/Home.jsx

```
function Home() {
  return (
    <div>
      <h2>Welcome to Sunrise Public School</h2>
      <p>
        Welcome to Sunrise public school we provide
        quality education to students
      </p>
      <h3>School News</h3>
      <p>
        Admissions are open for the academic year
        2026-2027
      </p>
    </div>
  );
}
export default Home;
```

In src/About.jsx

```
function About() {
  return (
    <div>
      <h2>About Our School</h2>
      <p>
        Sunrise public school was established in 1995.
        Our school provides education from primary to
        secondary level.
      </p>
      <p>
        Welcome, we focus on academics, sports, cultural
        activities and overall development of students.
      </p>
    </div>
  );
}
export default About;
```

In src/Classes.jsx

```
function Classes() {
  return (
```

```
      <div>
        <h2>Classes</h2>
        <ul>
          <li>Class 1</li>
          <li>Class 2</li>
          <li>Class 3</li>
          <li>Class 4</li>
        </ul>
      </div>
    );
}
export default Classes;
```

In src/Teachers.jsx

```
function Teachers() {
  return (
    <div>
      <h2>Our Teachers</h2>
      <table border="1">
        <thead>
```

```
    <tr>
      <th>Teacher Name</th>
      <th>Subject</th>
      <th>Class</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>ABC</td>
      <td>Maths</td>
      <td>9th</td>
    </tr>
    <tr>
      <td>DDR</td>
      <td>Science</td>
      <td>9th</td>
    </tr>
```

```
            <tr>
              <td>XYZ</td>
              <td>English</td>
              <td>10th</td>
            </tr>
          </tbody>
        </table>
      </div>
    );
}

export default Teachers;
```

In src/Contact.jsx

```
function Contact() {
  return (
    <div>
      <h2>Contact Us</h2>
      <p>
        <b>School:</b> Sunrise Public School
```

```
            </p>
            <p>
              <b>Address:</b> Mumbai, Maharashtra
            </p>
            <p>
              <b>Email:</b> Sunrise@gmail.com
            </p>
            <p>
              <b>Phone:</b> 987654321
            </p>
          </div>
    );
}
export default Contact;
```

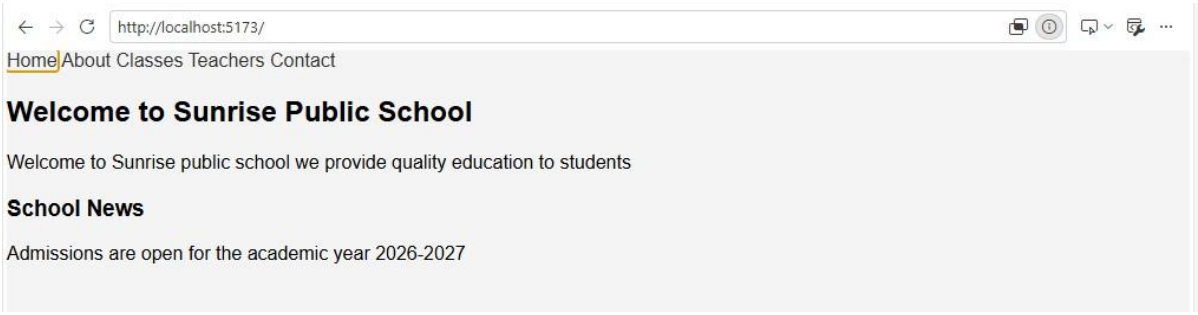
In src/App.jsx

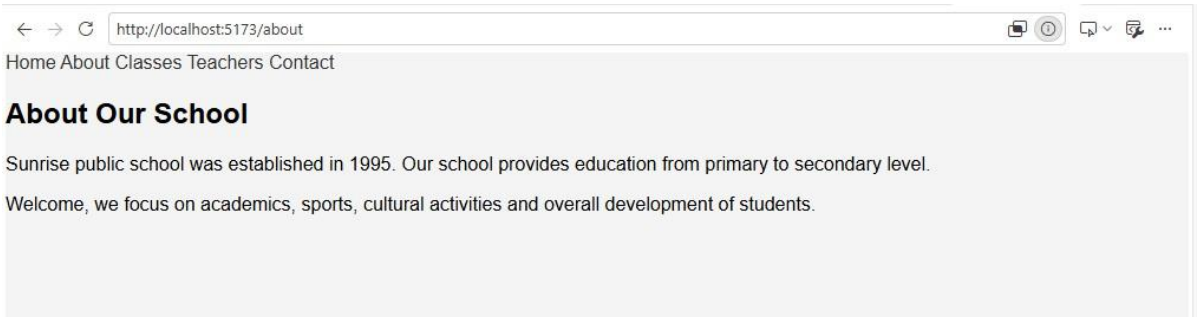
```
import React from "react";
import {
  BrowserRouter as Router,
  Routes,
  Route,
```

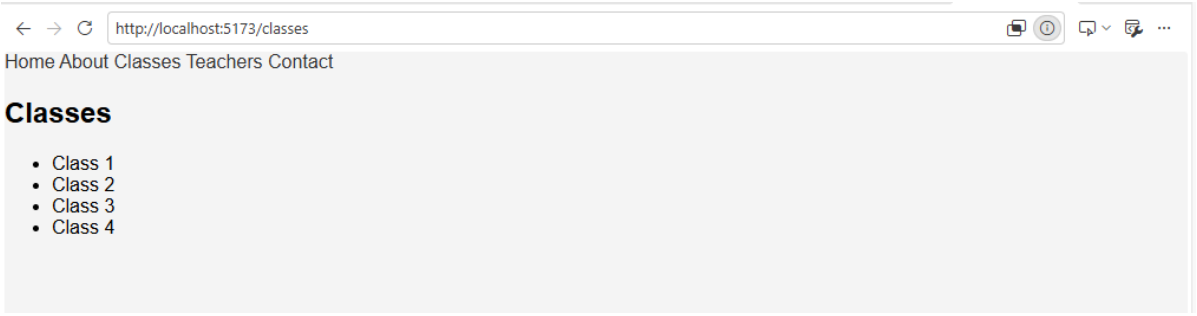
```
  Link
}
from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Classes from "./Classes";
import Teachers from "./Teachers";
import Contact from "./Contact";
function App() {
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link>{" "}
        <Link to="/about">About</Link>{" "}
        <Link to="/classes">Classes</Link>{" "}
        <Link to="/teachers">Teachers</Link>{" "}
        <Link to="/contact">Contact</Link>{" "}
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
```

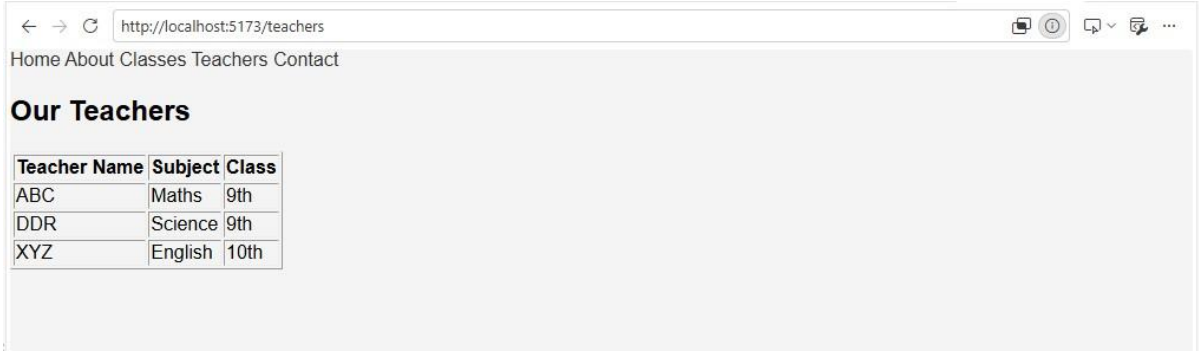
```
            <Route path="/about" element={<About />} />
            <Route path="/classes" element={<Classes />} />
            <Route path="/teachers" element={<Teachers />} />
            <Route path="/contact" element={<Contact />} />
          </Routes>
        </Router>
    );
}
export default App;
```

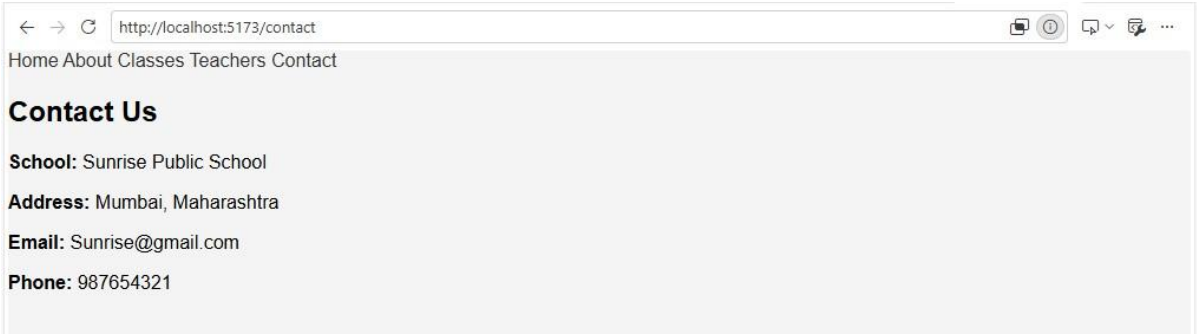
Output:-






http://localhost:5173/classes
Home About Classes Teachers Contact
Classes
Class 1
Class 2
Class 3
Class 4


http://localhost:5173/teachers
Home About Classes Teachers Contact
Our Teachers
Teacher Name Subject Class
ABC Maths 9th
DDR Science 9th
XYZ English 10th


http://localhost:5173/contact
Home About Classes Teachers Contact
Contact Us
School: Sunrise Public School
Address: Mumbai, Maharashtra
Email: Sunrise@gmail.com
Phone: 987654321

c) Use Link and Routes to move between pages.

In src/Services.jsx

```
function Services()
{
    return (
        <div>
          <h2>Our Services</h2>
           <ul>
             <li>Web Application Development</li>
              <li>Database Management</li>
               <li>API Development</li>
          </ul>
        </div>
   );
}
export default Services;
```

In src/Contact.jsx

```
function Contact()
 {
   return (
        <div>
           <h2>Contact Us</h2>
            <p>Email: support@example.com</p>
             <p>Phone: 9876543210</p>
        </div>
   );
```

```
}
export default Contact;
```

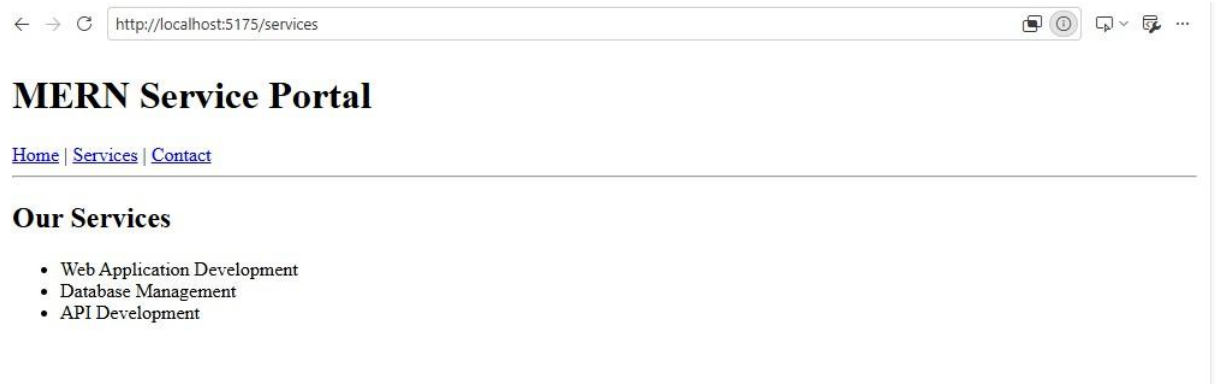
In src/App.jsx

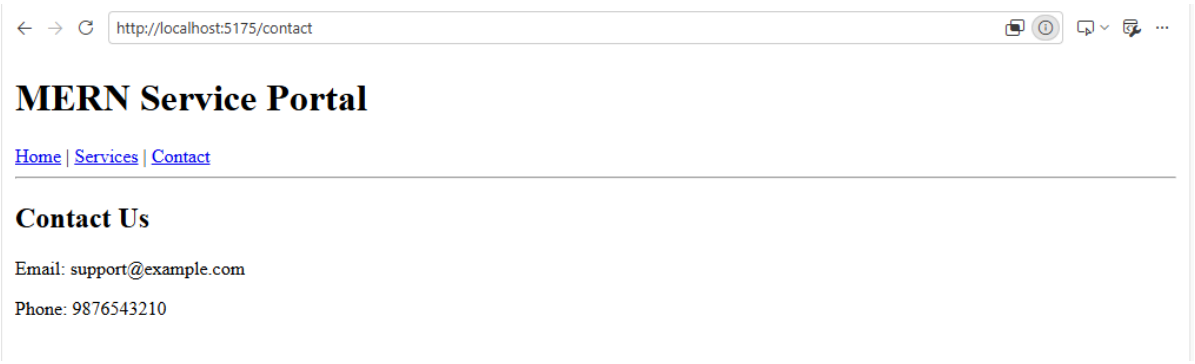
```
import { Routes, Route, Link } from "react-router-dom";
import Services from "./Services.jsx";
import Contact from "./Contact.jsx";
 function App()
  {
   return (
     <div>
      <h1>MERN Service Portal</h1>
       <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/services">Services</Link> |{" "}
        <Link to="/contact">Contact</Link>
       </nav>
       <hr />
        <Routes>
          <Route path="/" element={<h2>Welcome to Our Website</h2>} />
          <Route path="/services" element={<Services />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
   </div>
 );
}
```

```
export default App;
```

Output:-







d) Fetch data from public API using Axios.

Axios:- Axios is a popular promised based Http client for JavaScript. It is commonly used to make API request in browser and node.js.

To install the Axios we used the command:

- npm install axios
- npm start

In src/App.jsx

```
import React, { useEffect, useState } from "react";
import axios from "axios";
function App() {
  const [products, setProducts] = useState([]);
  useEffect(() => {
    axios.get("https://dummyjson.com/products")
      .then((response) => {
        setProducts(response.data.products);
      })
      .catch((error) => {
        console.log(error);
      });
  }, []);
  return (
    <div>
      <h1>Product Details</h1>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Product Name</th>
```

```
                <th>Price</th>
                <th>Category</th>
                <th>Rating</th>
              </tr>
            </thead>
            <tbody>
              {products.map((product) => (
                <tr key={product.id}>
                  <td>{product.id}</td>
                  <td>{product.title}</td>
                  <td>{product.price}</td>
                  <td>{product.category}</td>
                  <td>{product.rating}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
    );
}
export default App;
```

Output:-

http://localhost:5173/

Product Details

ID	Product Name	Price	Category	Rating
1	Essence Mascara Lash Princess	9.99	beauty	2.56
2	Eyeshadow Palette with Mirror	19.99	beauty	2.86
3	Powder Canister	14.99	beauty	4.64
4	Red Lipstick	12.99	beauty	4.36
5	Red Nail Polish	8.99	beauty	4.32
6	Calvin Klein CK One	49.99	fragrances	4.37
7	Chanel Coco Noir Eau De	129.99	fragrances	4.26
8	Dior J'adore	89.99	fragrances	3.8
9	Dolce Shine Eau de	69.99	fragrances	3.96
10	Gucci Bloom Eau de	79.99	fragrances	2.74
11	Annibale Colombo Bed	1899.99	furniture	4.77
12	Annibale Colombo Sofa	2499.99	furniture	3.92
13	Bedside Table African Cherry	299.99	furniture	2.87
14	Knoll Saarinen Executive Conference Chair	499.99	furniture	4.88
15	Wooden Bathroom Sink With Mirror	799.99	furniture	3.59
16	Apple	1.99	groceries	4.19
17	Beef Steak	12.99	groceries	4.47
18	Cat Food	8.99	groceries	3.13
19	Chicken Meat	9.99	groceries	3.19
20	Cooking Oil	4.99	groceries	4.8
21	Cucumber	1.49	groceries	4.07
22	Dog Food	10.99	groceries	4.55
23	Eggs	2.99	groceries	2.53
24	Fish Steak	14.99	groceries	3.78
25	Green Bell Pepper	1.29	groceries	3.25
26	Green Chili Pepper	0.99	groceries	3.66
27	Honey Jar	6.99	groceries	3.97
28	Ice Cream	5.49	groceries	3.39
29	Juice	3.99	groceries	3.94
30	Kiwi	2.49	groceries	4.93

e) Display fetched data in table or list format.

In src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";
function App() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Unable to retrieve users:", error);
      });
  }, []);
  return (
    <div className="container">
      <h1>User Information</h1>
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
```

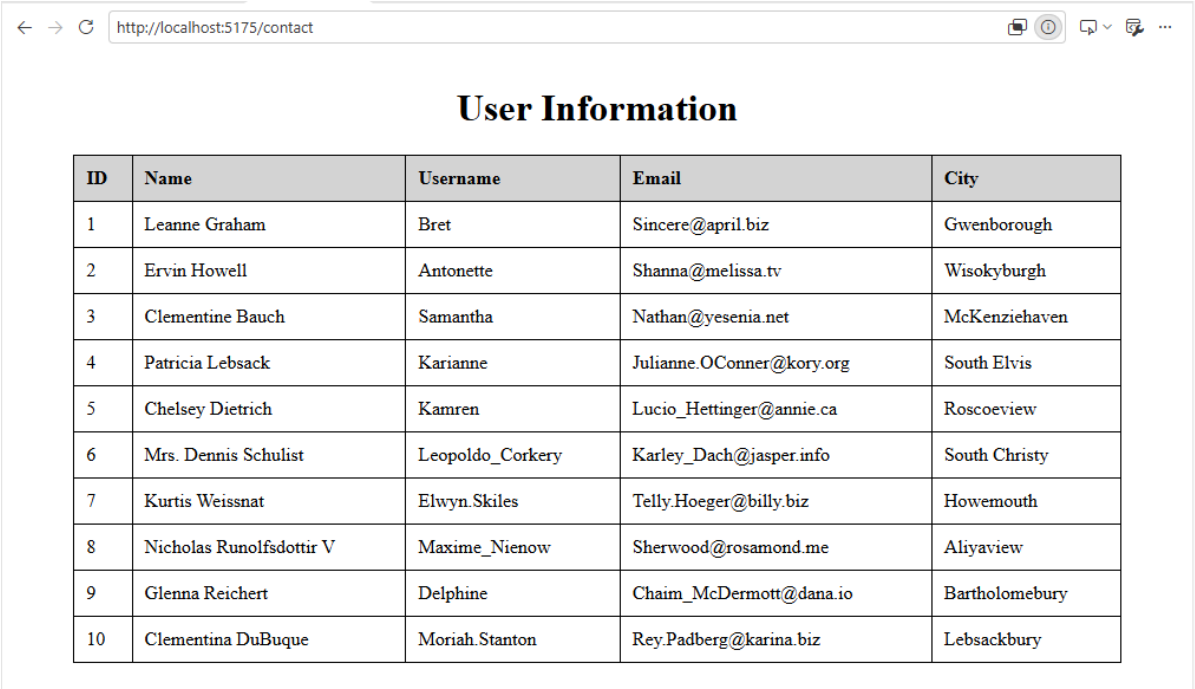
```
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.username}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```

In src/App.css

```
.container {
  width: 90%;
  margin: 30px auto;
  text-align: center;
```

```
}
table {
  width: 100%;
  border-collapse: collapse;
}
th,
td {
  border: 1px solid black;
  padding: 10px;
  text-align: left;
}
th {
  background-color: lightgray;
}
```

Output:-



ID	Name	Username	Email	City
1	Leanne Graham	Bret	Sincere@april.biz	Gwenborough
2	Ervin Howell	Antonette	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Samantha	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Leopoldo_Corkery	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Elwyn.Skiles	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Maxime_Nienow	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Delphine	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Moriah.Stanton	Rey.Padberg@karina.biz	Lebsackbury

f) Implement search or filter functionality on displayed data.

In src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";
function App() {
  const [users, setUsers] = useState([]);
  const [searchText, setSearchText] = useState("");
  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Unable to retrieve users:", error);
      });
  }, []);
  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(searchText.toLowerCase())
  );
  return (
    <div className="container">
      <h1>Search User</h1>
      <input
        type="text"
```

```
        placeholder="Enter user name"
        value={searchText}
        onChange={(event) => setSearchText(event.target.value)}
      />
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

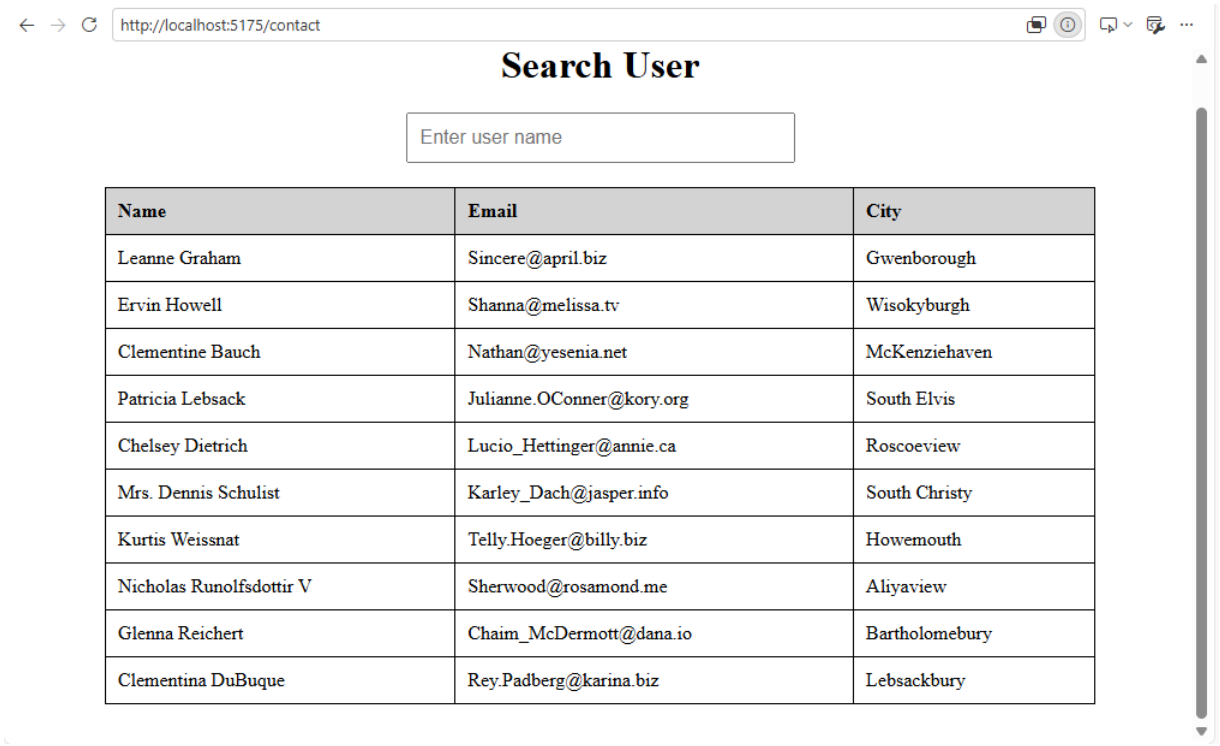
```
export default App;
```

In src/App.css

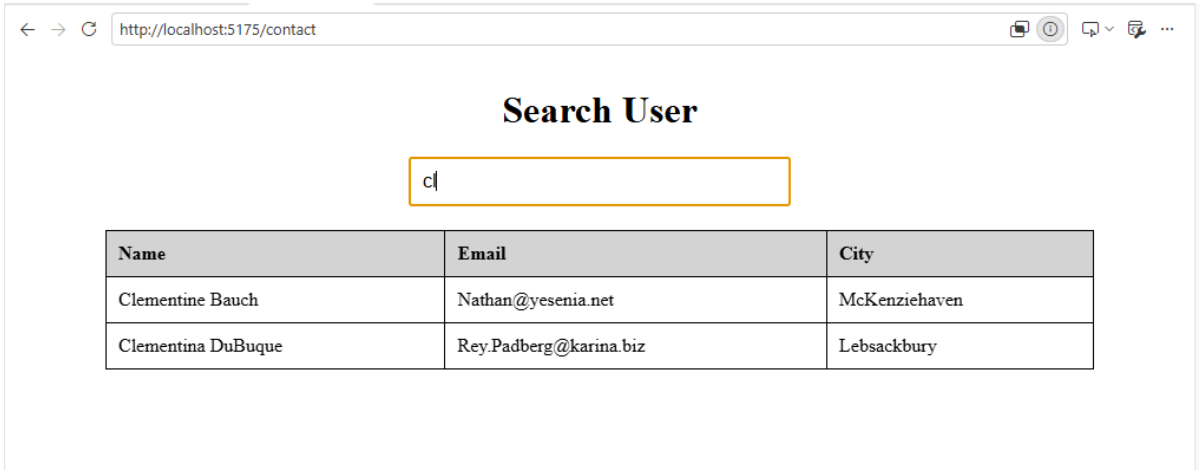
```
.container {
  width: 85%;
  margin: 30px auto;
  text-align: center;
}
input {
  width: 300px;
  padding: 10px;
  margin-bottom: 20px;
  font-size: 16px;
}
table {
  width: 100%;
  border-collapse: collapse;
}
th,
td {
  border: 1px solid black;
  padding: 10px;
  text-align: left;
}
th {
  background-color: lightgray;
```

}

Output:-



Name	Email	City
Leanne Graham	Sincere@april.biz	Gwenborough
Ervin Howell	Shanna@melissa.tv	Wisokyburgh
Clementine Bauch	Nathan@yesenia.net	McKenziehaven
Patricia Lebsack	Julianne.OConner@kory.org	South Elvis
Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview
Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy
Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth
Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview
Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury
Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury



Name	Email	City
Clementine Bauch	Nathan@yesenia.net	McKenziehaven
Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury

Module-2

Practical – 6

Aim: Node.js and Express Fundamentals

a. Install Express and create a simple server.

Procedure Since this is the first Express practical, the initial project setup and Express installation will be explained step by step.

1: Create the Practical Folder

Inside the MERN Practicals folder, create a new folder named: Practical-1. Open this folder in Visual Studio Code.

2: Open the Integrated Terminal

Select: Terminal -> New Terminal

The terminal should display a path similar to:

PS D:\MERN_Practicals\Practical-6>

```
D:\MERN_Practicals>cd Practical-6

D:\MERN_Practicals\Practical-6>
```

3 : Initialize the Node.js Project

Execute the init command in the terminal.

This command creates a package.json file containing the basic information and dependencies of the Node.js project.

```
D:\MERN_Practicals\Practical-6>npm init -y
Wrote to D:\MERN_Practicals\Practical-6\package.json:

{
  "name": "practical-6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

4: Install Express.js

Execute

The install command for express.(npm install express)

After successful installation, Express will be added to the project dependencies.

```
D:\MERN_Practicals\Practical-6>npm install express

added 68 packages, and audited 69 packages in 5s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

5: Create the Server File

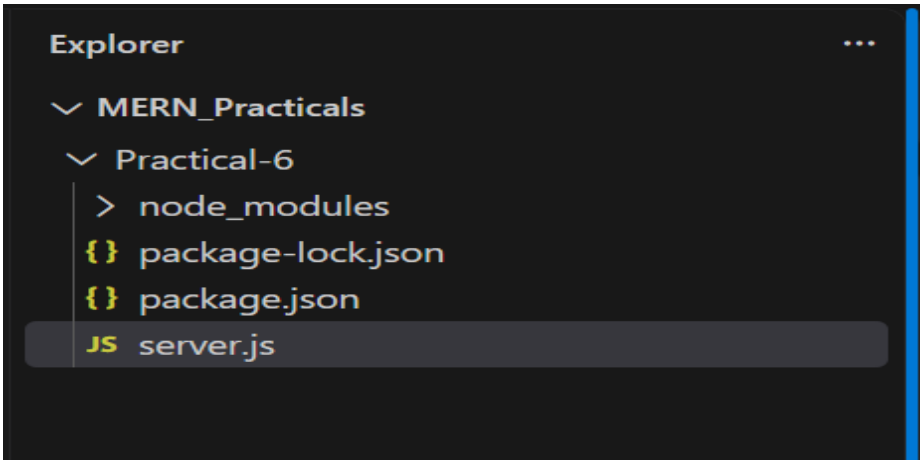
Create a new file named: server.js

The project structure will appear similar to:

Practical-6 containing

- node_modules
- package-lock.json
- package.json

server.js



6: Write the Express Server Program

Open server.js and enter the server program.

```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/", (req, res) => {
res.send("Welcome to MERN Stack Backend Development");
});
app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

7: Run the Server

Save the file and execute the file using node.

Node server.js

The terminal displays: Server is running on port 3000

```
D:\MERN_Practicals\Practical-6>node server.js
Server running on port 3000
```

8: Open the Application in the Browser

Open the following address in the browser:

http://localhost:3000

The browser displays the response sent by the Express server

Output:



Welcome to MERN Stack Backend Development

b. Create Multiple Routes in Express

Procedure:

Create multiple routes like /, /about, /courses, /contact in server.js

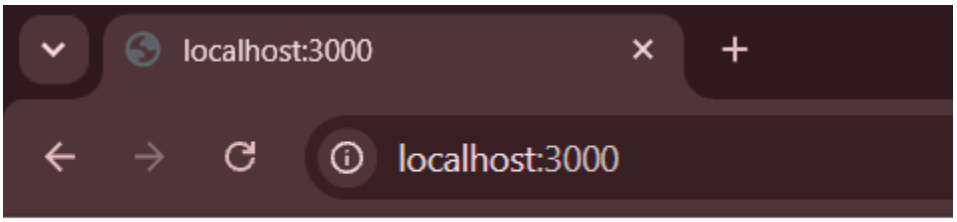
```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/", (req, res) => {
   res.send("Welcome to the Home Page");
});
app.get("/about", (req, res) => {
   res.send("About MERN Stack Development");
});
app.get("/courses", (req, res) => {
  res.send("Course: B.Sc. Information Technology");
});
app.get("/contact", (req, res) => {
  res.send("Contact: mern@example.com");
});
app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

run the server.

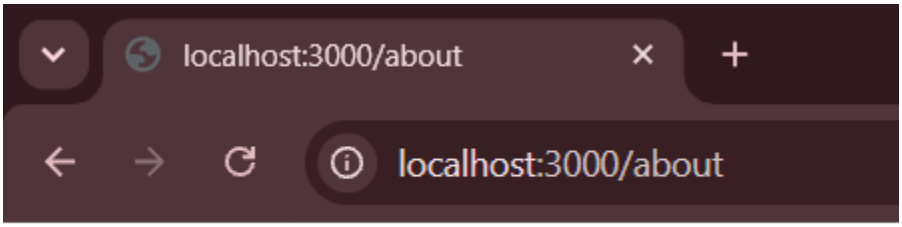
```
Node server.js
```

Open the URLs individually in browser to see output.

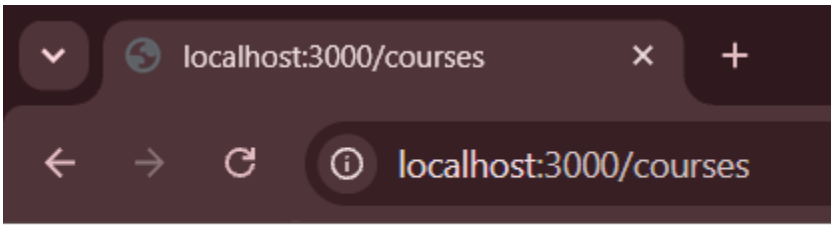
Output:



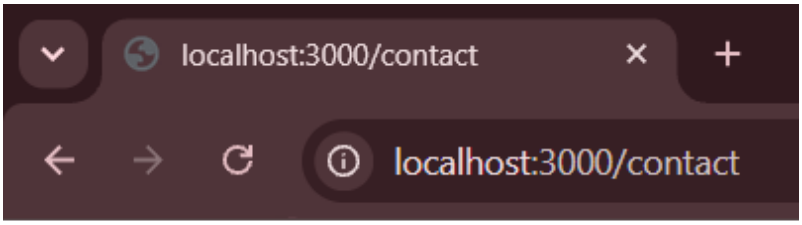
Welcome to the Home Page



About MERN Stack Development



Course: B.Sc. Information Technology



Contact: mern@example.com

C. Send HTML Content from Express Server

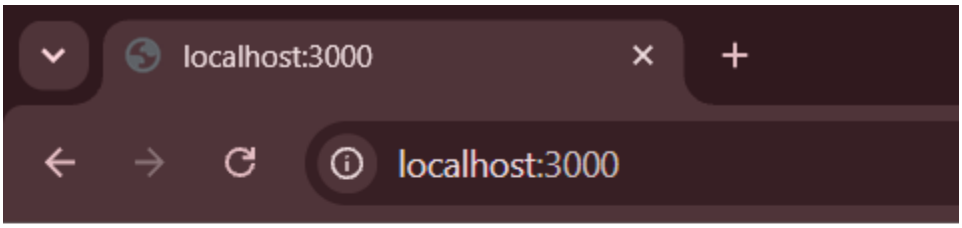
Procedure:

Create a route that sends HTML content in server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/", (req, res) => {
res.send(`
    <h1>MERN Stack Development</h1>
    <h2>Backend Using Express.js</h2>
    <p>Welcome to the Express server</p>
    <p>Course: B.Sc. Information Technology</p>
    <p>Semester: V</p>
  `);
});
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

open local host:3000

Output:



MERN Stack Development

Backend Using Express.js

Welcome to the Express server.

Course: B.Sc. Information Technology

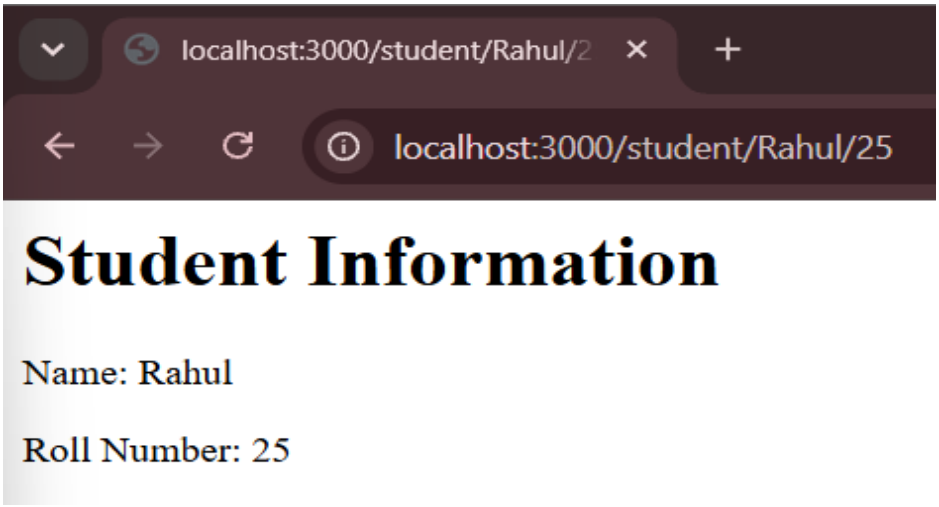
Semester: V

d. Handle Route Parameters in Express

in server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/student/:name/:rollno", (req, res) => {
    const studentName = req.params.name;
    const rollNo = req.params.rollno;
    res.send(`
      <h1>Student Information</h1>
      <p>Name: ${studentName}</p>
      <p>Roll Number: ${rollNo}</p>
`);
});
app.listen(PORT, () => {
     console.log(`Server running on port ${PORT}`);
});
```

Output:

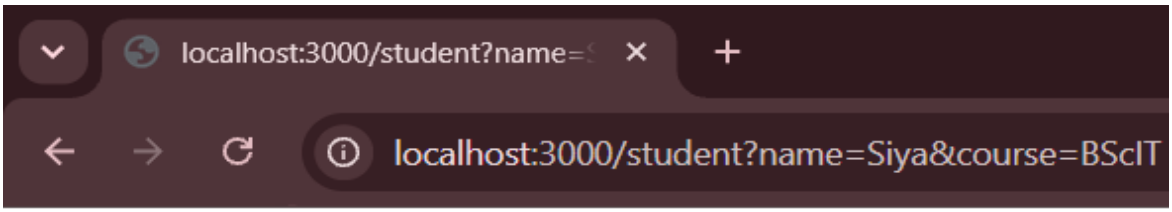


e. Handle Query Parameters in Express

in server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/student", (req, res) => {
        const name = req.query.name;
        const course = req.query.course;
        res.send(`
          <h1>Student Details</h1>
          <p>Name: ${name}</p>
          <p>Course: ${course}</p>
        `);
});
app.listen(PORT, () => {
     console.log(`Server running on port ${PORT}`);
});
```

Output: http://localhost:3000/student?name=Siya&course=BScIT



Student Details

Name: Siya

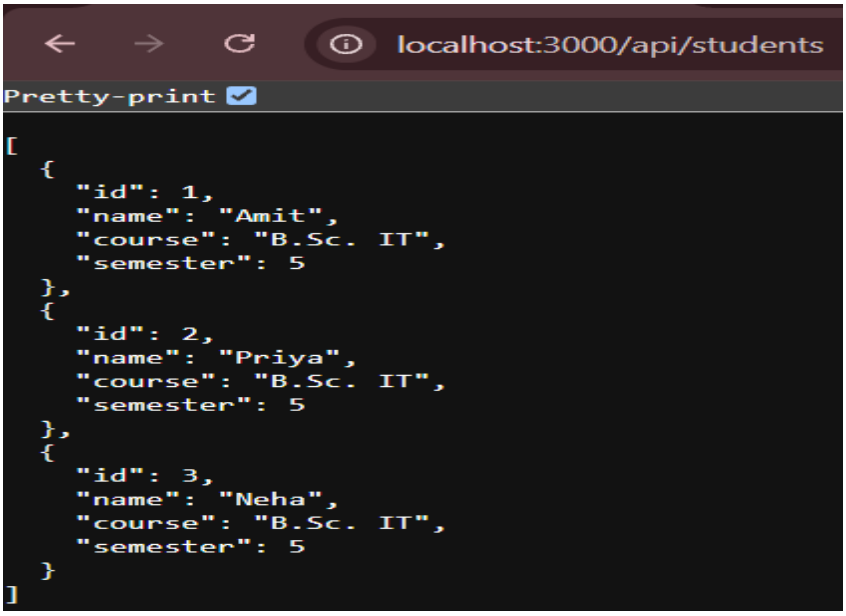
Course: BScIT

f. Create a Simple JSON API using Express

in server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
const students = [
    { id: 1, name: "Amit", course: "B.Sc. IT", semester: 5 },
    { id: 2, name: "Priya", course: "B.Sc. IT", semester: 5 },
    { id: 3, name: "Neha", course: "B.Sc. IT", semester: 5 }
];
app.get("/api/students", (req, res) => {
    res.json(students);
});
app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

Output: http://localhost:3000/api/students



Practical-7

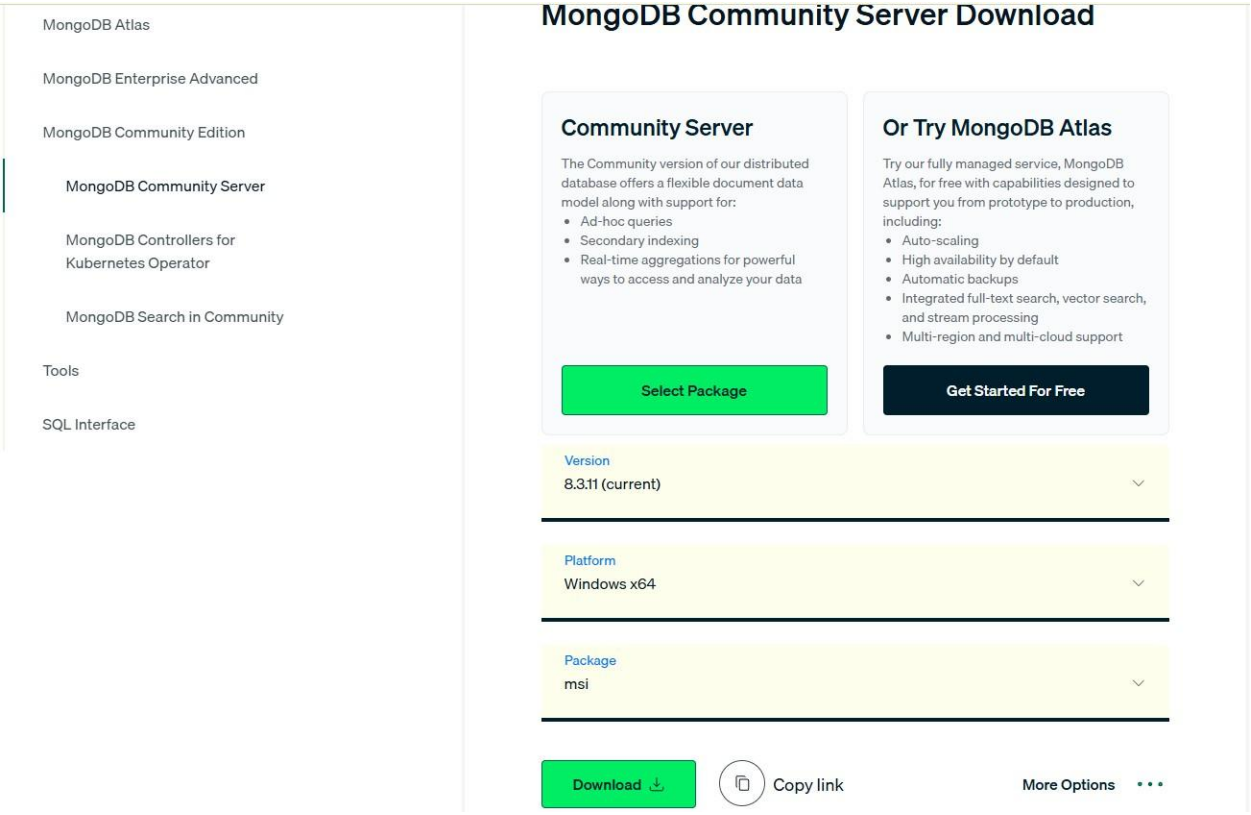
Aim: Install MongoDB Community Server and MongoDB Compass.

MongoDB is a NoSQL, document-oriented database used in MERN Stack.

Unlike SQL databases where data is stored in tables and rows, MongoDB stores data in collections and documents

a. Download MongoDB Community Server

https://www.mongodb.com/try/download/community



Choose Complete installation Install MongoDB Compass

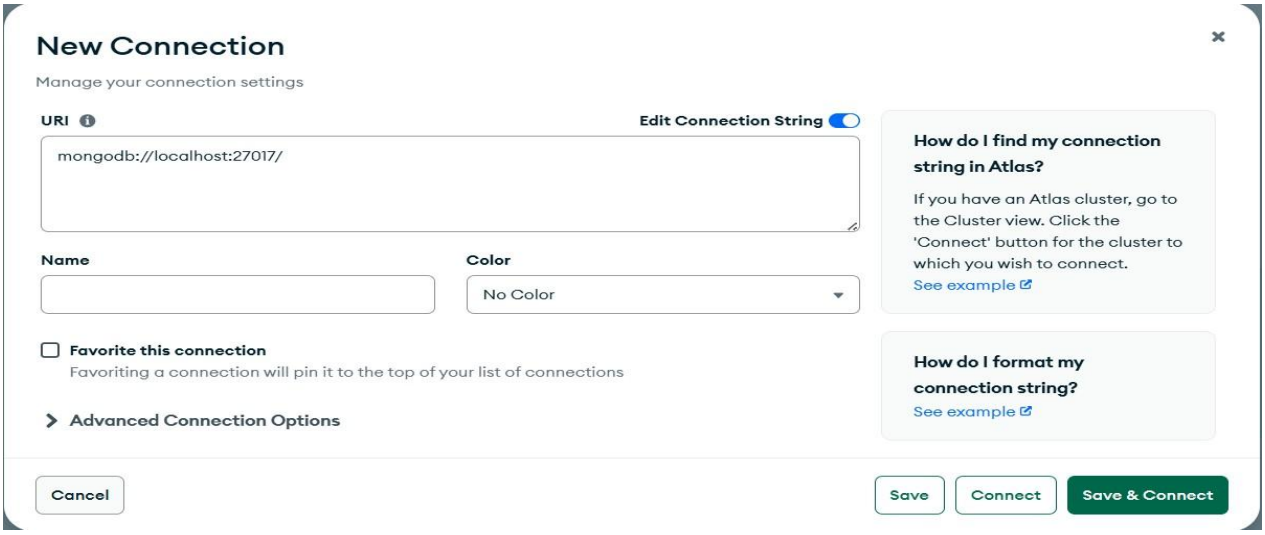
Finish

Open MongoDB Compass Launch MongoDB Compass.

The default connection string appears as:

mongodb://localhost:27017

Click save & connect



Create a Database

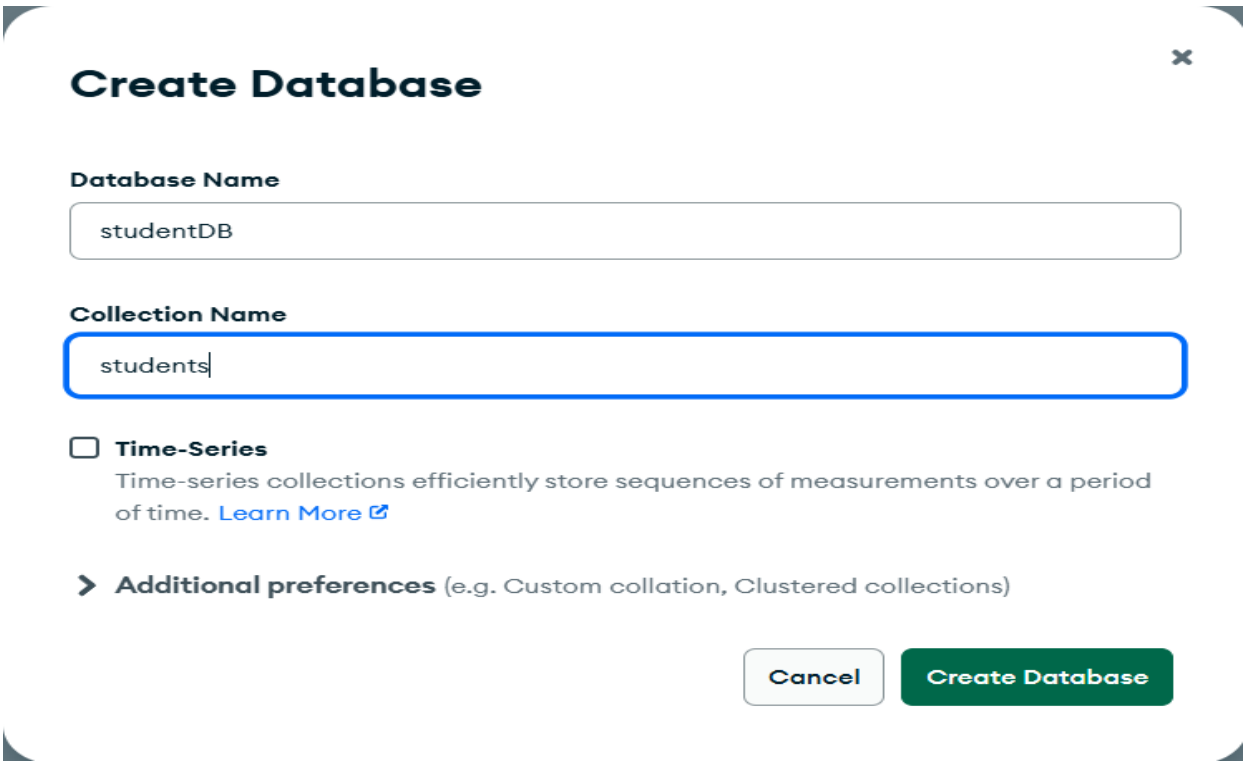
Click Create Database

Enter:

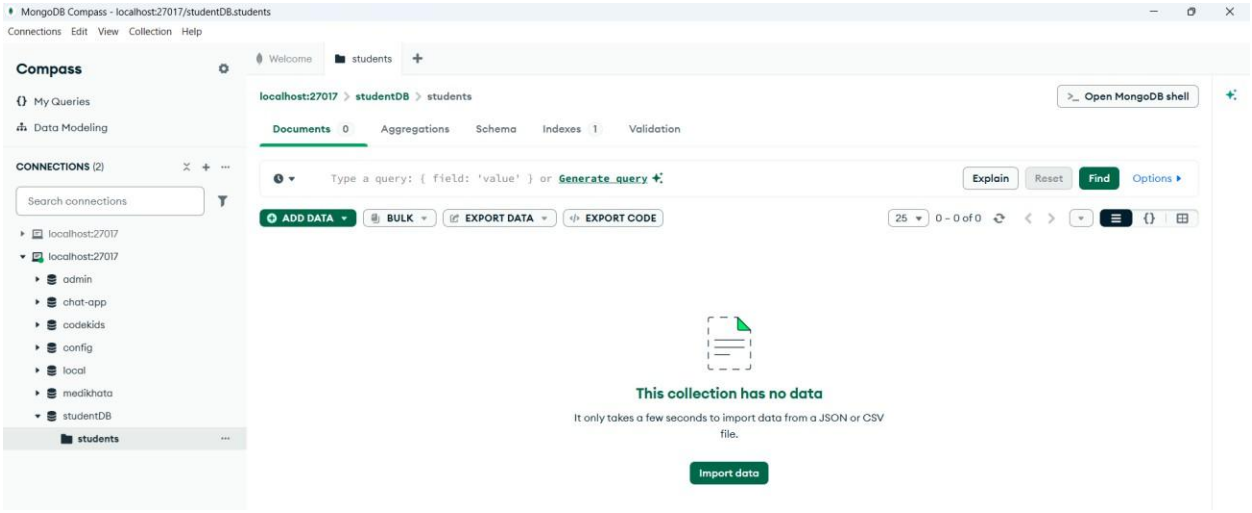
Database Name = studentDB

Collection Name = students

Click Create Database



Creating the studentDB Database



b. Perform CRUD operations using MongoDB Compass.

Program / Operations

Insert the following document into the students collection.

{

"studentId": 101,

"name": "Rahul Sharma",

"course": "M.Sc. Information Technology",

"semester": 3,

"city": "Delhi"

}

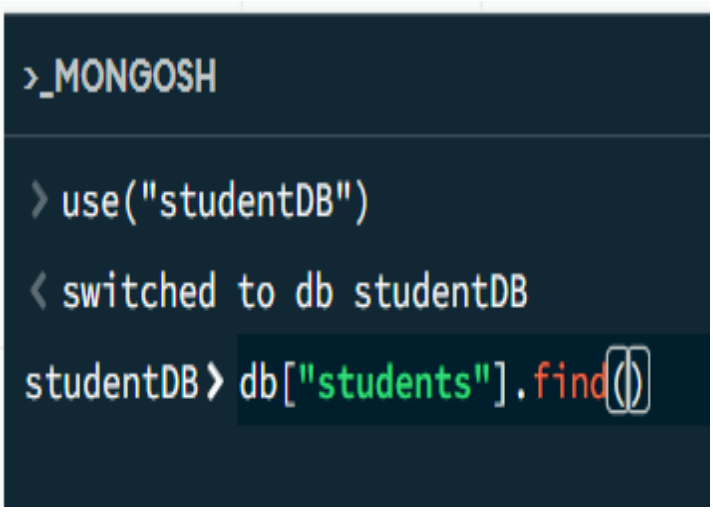
Perform the following operations:

Insert a document

View documents

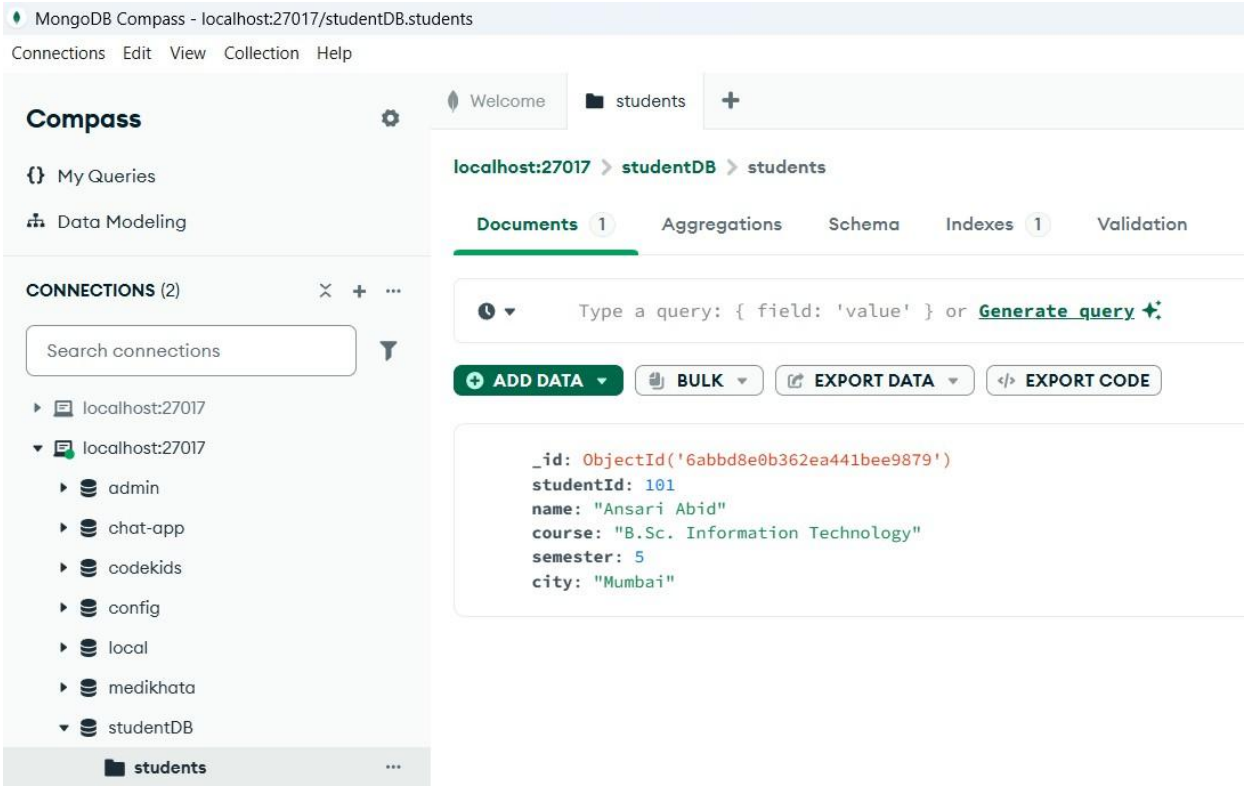
Update the city field

Delete the document



```
>_MONGOSH
> use("studentDB")
< switched to db studentDB
> db["students"].find()
<
> db.students.insertOne({ studentId:101, name:"Ansari Abid", course:"B.Sc. Information Technology", semester:5, city:"Mumbai"  })
< {
    acknowledged: true,
    insertedId: ObjectId('6abbd8e0b362ea441bee9879')
  }
> db.students.find()
< {
    _id: ObjectId('6abbd8e0b362ea441bee9879'),
    studentId: 101,
    name: 'Ansari Abid',
    course: 'B.Sc. Information Technology',
    semester: 5,
    city: 'Mumbai'
  }
studentDB >
```

Output:



c. Connect Node.js with MongoDB using Mongoose.

Create a folder named MONGO

Open the integrated terminal

Bash

> Cd MONGO

>npm init –y

>npm install mongoose

```
Problems  Output  Debug Console  Terminal  Ports

D:\MERN_Practicals>cd mongo

D:\MERN_Practicals\mongo>npm init -y
Wrote to D:\MERN_Practicals\mongo\package.json:

{
  "name": "mongo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
D:\MERN_Practicals\mongo>npm install mongoose

added 19 packages, and audited 21 packages in 5s

1 package is looking for funding
  run `npm fund` for details

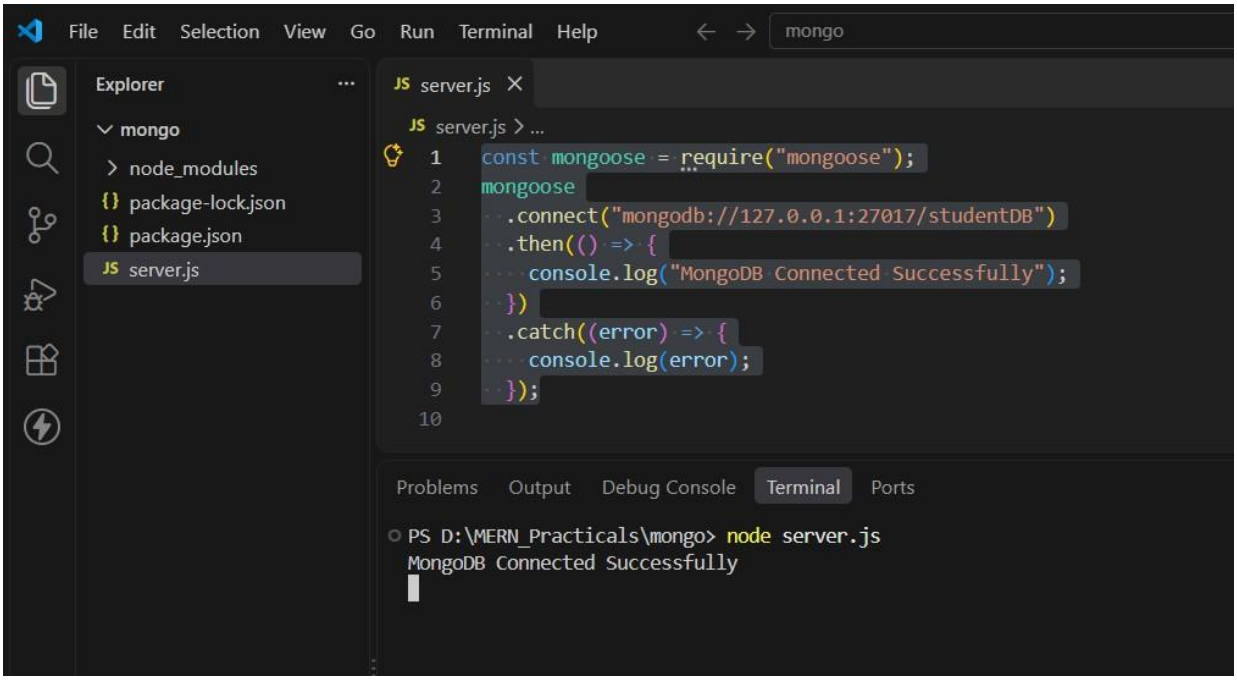
found 0 vulnerabilities
```

Create a file inside MONGO folder named server.js

```
const mongoose = require("mongoose");
mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log(error);
  });
```

Run:

Node server.js



Practical-8

Aim: API Creation and Integration

a. Create a MongoDB Scheme and model using Mongoose.

Procedure

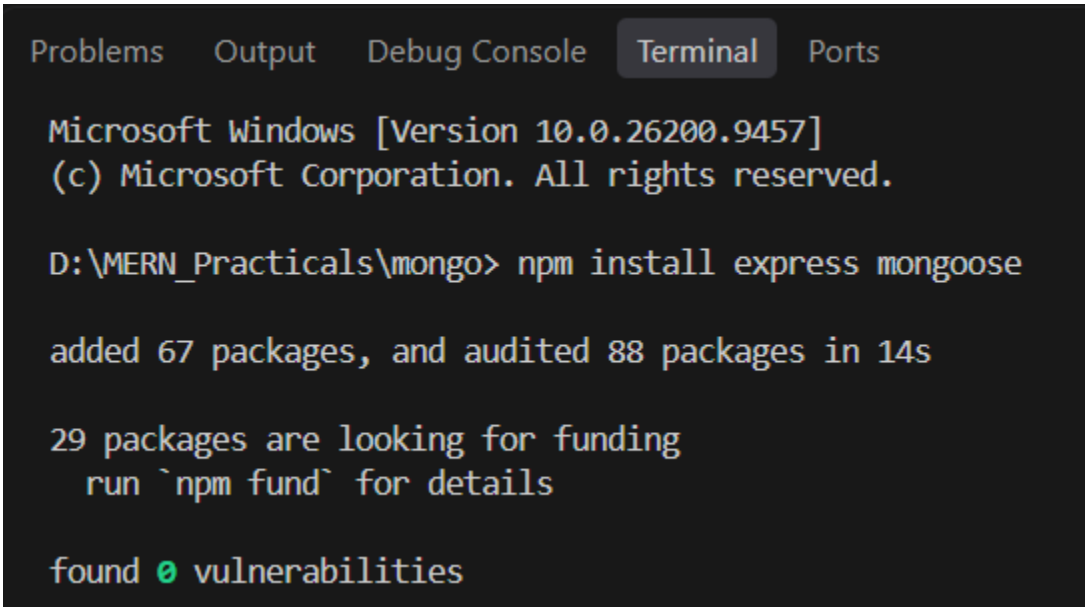
1. Open the Practical-3 folder in Visual Studio Code.

2 Open the integrated terminal and initialize the project.

npm init –y

3 Install the required packages.

npm install express mongoose



4. Create a file named

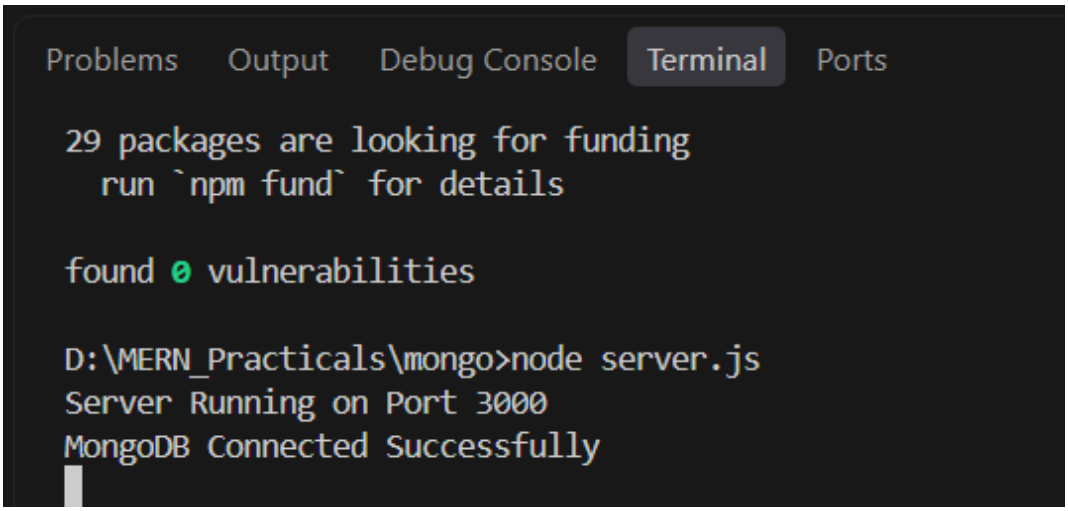
Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
app.use(express.json());
mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
```

```
  .then(() => console.log("MongoDB Connected Successfully"))
  .catch((err) => console.log(err));
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  semester: Number,
  city: String,
});
const Student = mongoose.model("Student", studentSchema);
app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

Run

>node server.js



b. Implement REST API Operations.

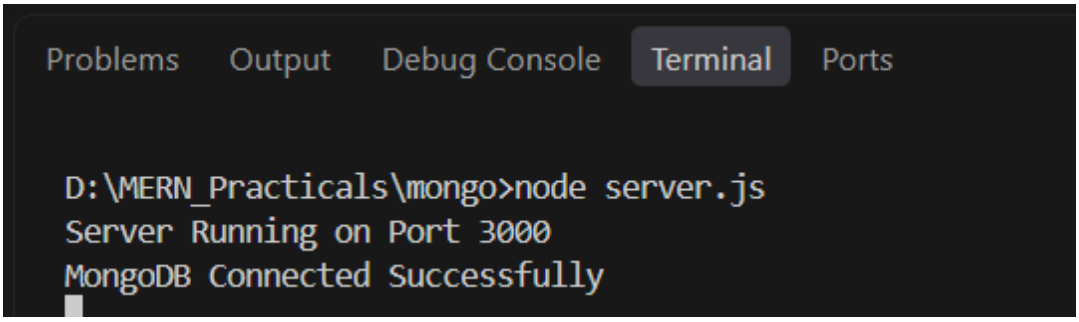
In server.js

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
app.use(express.json());
mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => console.log("MongoDB Connected Successfully"))
  .catch((err) => console.log(err));
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  semester: Number,
  city: String,
});
const Student = mongoose.model("Student", studentSchema);
// POST
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  await student.save();
  res.json(student);
});
// GET
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
```

```
});
// PUT
app.put("/students/:id", async (req, res) => {
  const student = await Student.findByIdAndUpdate(req.params.id, req.body, {
    new: true,
  });
  res.json(student);
});
// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({
    message: "Student Deleted Successfully",
  });
});
app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

Output:

Execute: node server.js



c. Test REST APIs Using Postman.

Procedure

Install Postman

1 Open your web browser and visit:

postman.com

https://www.postman.com/downloads/

2 Download the Windows 64-bit version.

3 Run the downloaded installer.

4 Follow the installation wizard and click Install.

5 After installation open Postman.

Skip Login (Recommended for Students)

When Postman opens, it may ask you to sign in.

Choose one of these options:

Skip and go to the app

Continue without an account (Depending on the version, the wording may differ.)

Create Your First Request

Click New or the + tab.

Select HTTP Request.

At the top, change the request method to GET.

Enter the URL:

localhost:3000

Click Send.

If your Express server is running, you will see the response from your API.

Test POST Request

Change the method to POST.

Enter the URL: localhost:3000/students

Click Body.

Select raw.

From the dropdown, choose JSON

Enter:

```
{
  "name": "Rahul Sharma",
  "course": "B.Sc. Information Technology",
  "semester": 5,
  "city": "Mumbai"
}
```

Click Send.

If successful, the inserted document will be displayed in the response.

Open Postman and test the following requests

Method	**Url**
GET	http://localhost:3000/students
PUT	http://localhost:3000/students/:id
DELETE	http://localhost:3000/students/:id

Output

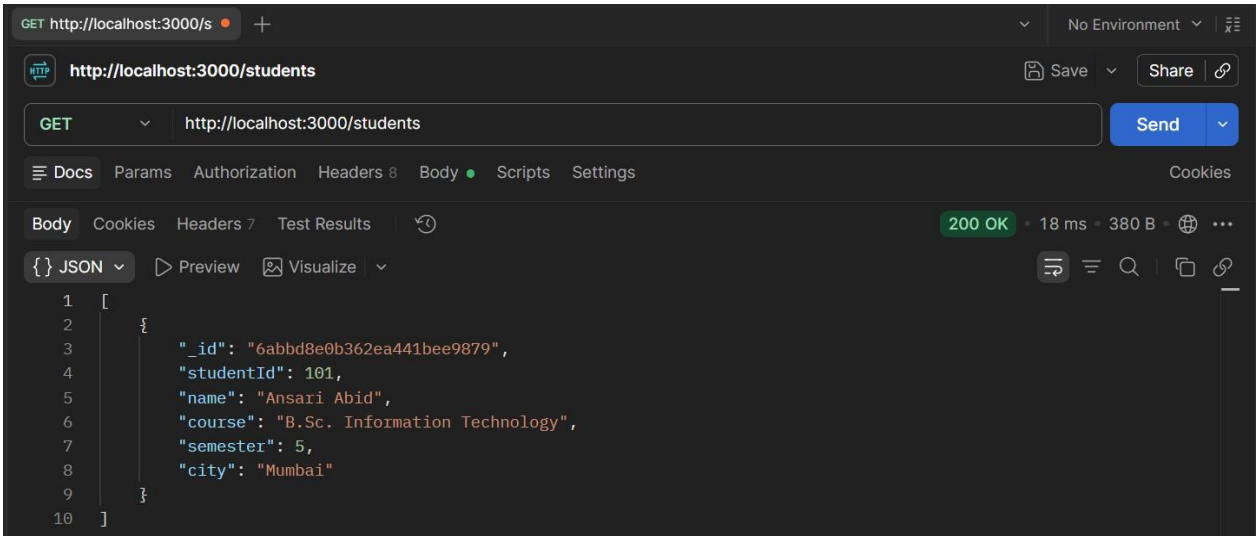
Perform the following operations in Postman:

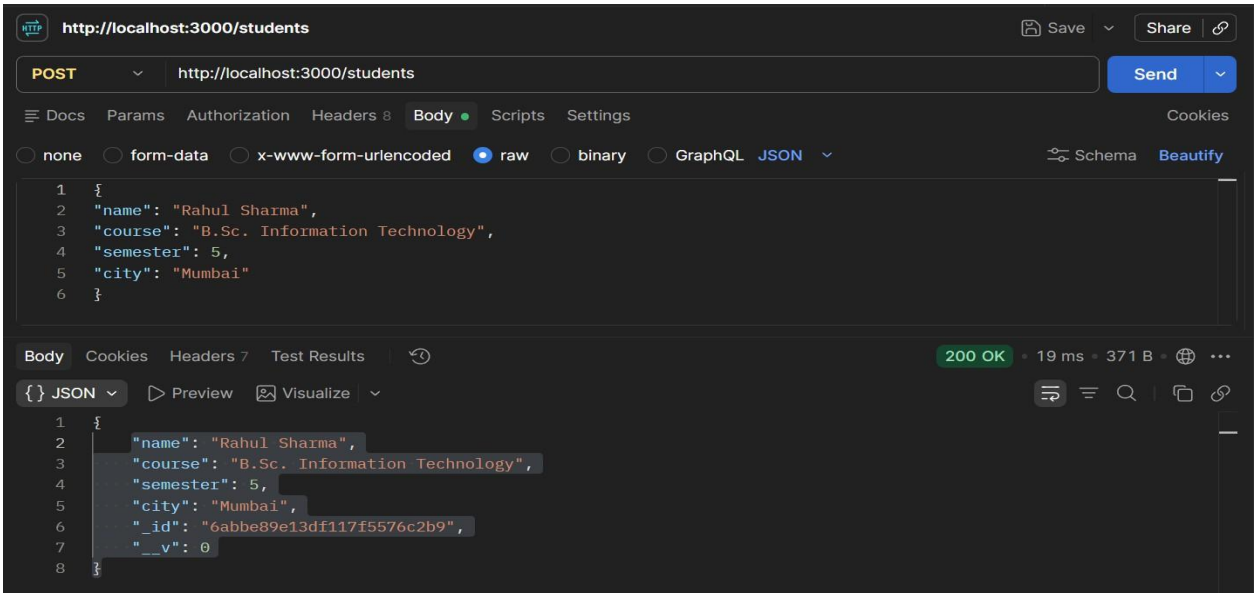
Insert a student record.

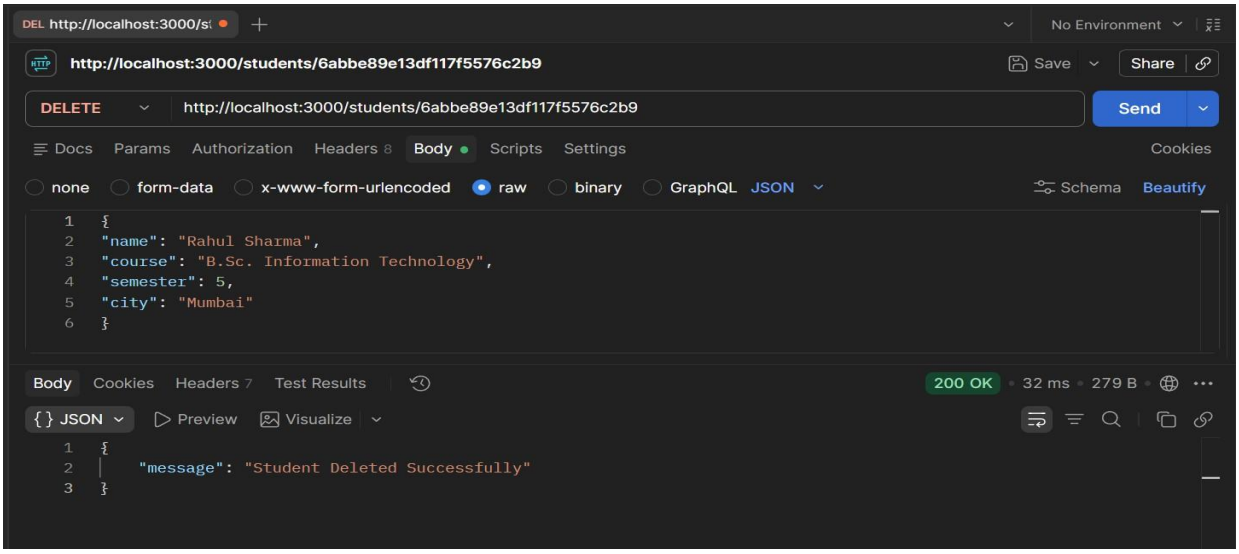
Retrieve all student records.

Update a student record.

Delete a student record


GET http://localhost:3000/s
No Environment
http://localhost:3000/students
Save
Share
GET
http://localhost:3000/students
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
Body Cookies Headers 7 Test Results
200 OK 18 ms 380 B
{ } JSON Preview Visualize
[
{
"_id": "6abbd8e0b362ea441bee9879",
"studentId": 101,
"name": "Ansari Abid",
"course": "B.Sc. Information Technology",
"semester": 5,
"city": "Mumbai"
}
]


http://localhost:3000/students
Save
Share
POST
http://localhost:3000/students
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Rahul Sharma",
"course": "B.Sc. Information Technology",
"semester": 5,
"city": "Mumbai"
}
Body Cookies Headers 7 Test Results
200 OK 19 ms 371 B
{ } JSON Preview Visualize
{
"name": "Rahul Sharma",
"course": "B.Sc. Information Technology",
"semester": 5,
"city": "Mumbai",
"_id": "6abbe89e13df117f5576c2b9",
"__v": 0
}


DEL http://localhost:3000/s
No Environment
http://localhost:3000/students/6abbe89e13df117f5576c2b9
Save
Share
DELETE
http://localhost:3000/students/6abbe89e13df117f5576c2b9
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Rahul Sharma",
"course": "B.Sc. Information Technology",
"semester": 5,
"city": "Mumbai"
}
Body Cookies Headers 7 Test Results
200 OK 32 ms 279 B
{ } JSON Preview Visualize
{
"message": "Student Deleted Successfully"
}

Practical- 9

Aim: Frontend and Backend Integration using React and Axios.

Axios is used to send HTTP requests from React to the Express API. The backend also requires CORS middleware so that the React application running on one port can communicate with the Express server running on another port. (npm)

a. Create the React Frontend and connect it with Express.

To create a React application and connect it to the Express backend using Axios.Procedure

1: Keep the Backend Server Ready

Open the existing Practical_7 folder in one VS Code window.

Make sure the backend server.js contains the CRUD API developed in Practical 7.

Install CORS in the backend folder:

npm install cors

Add this line near the top of server.js:

const cors = require("cors");

Add the following middleware after creating the Express application:

app.use(cors());

app.use(express.json());

The beginning of the backend file should appear as follows:

```
const express = require("express");

const mongoose = require("mongoose");

const cors=require("cors");

const app = express();

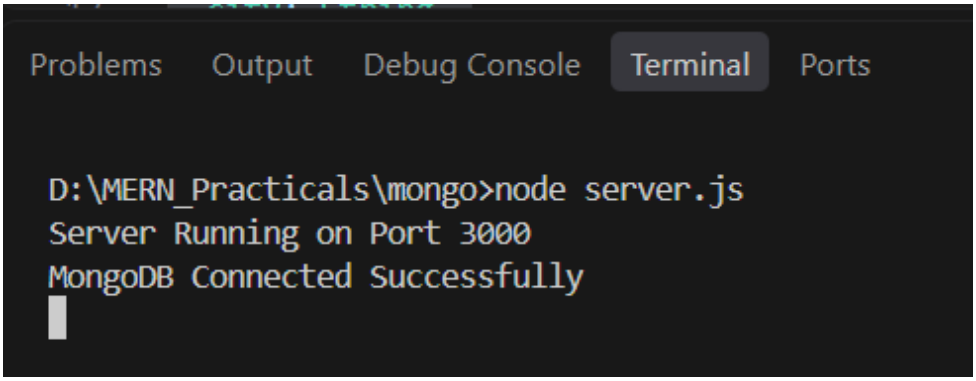

app.use(cors())

app.use(express.json());

mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => console.log("MongoDB Connected Successfully"))
```

```
  .catch((err) => console.log(err));
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  semester: Number,
  city: String,
});
const Student = mongoose.model("Student", studentSchema);
app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

Run the Backend: node server.js



2. Create the React Application

Open another VS Code window.

Create a folder named: Practical-9

Open the terminal and execute: npm create vite@latest

Enter the project name: student-frontend

Select:

Framework: React

Variant: JavaScript

Then execute: cd student-frontend

npm install

Vite provides a quick way to create a React project using a React template. (vitejs)

3. Install Axios

Inside the student-frontend folder,

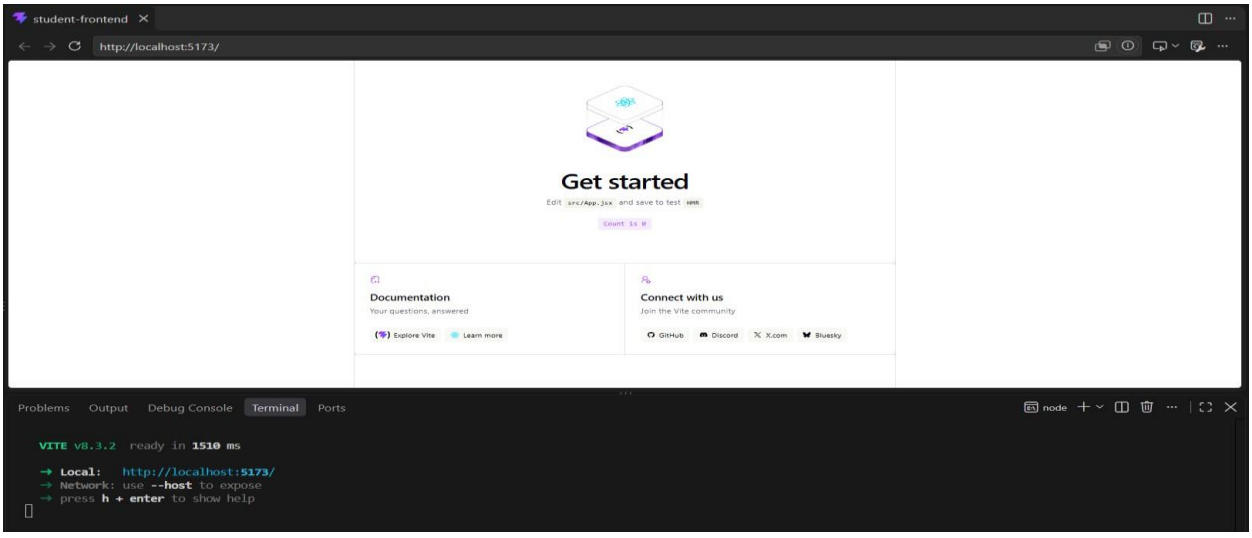
run: npm install axios

Axios is a promise-based HTTP client used to send requests such as GET, POST, PUT, and DELETE. (npm)

4: Start the React Application

Execute: npm run dev

Open the displayed local URL: http://localhost:51734/



B. Retrieve and Display Student Records.

To retrieve student records from the Express backend and display them in a React application.

In App.jsx

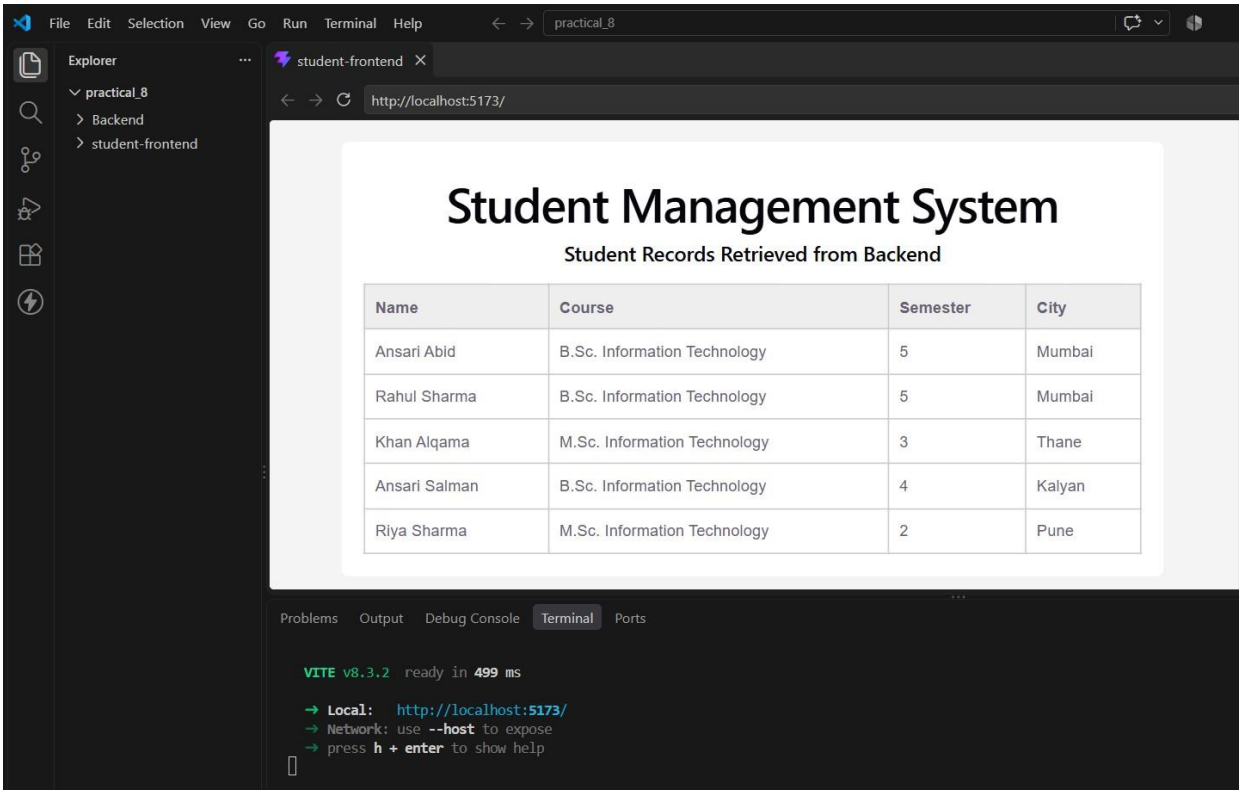
```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";
function App() {
  const [students, setStudents] = useState([]);
  const fetchStudents = async () => {
    try {
      const response = await axios.get("http://localhost:3000/students");
      setStudents(response.data);
    } catch (error) {
      console.error("Error fetching students:", error);
    }
  };
  useEffect(() => {
    fetchStudents();
  }, []);
  return (
    <div className="container">
      <h1>Student Management System</h1>
      <h2>Student Records Retrieved from Backend</h2>
      <table>
        <thead>
          <tr>
            <th>Name</th>
```

```
            <th>Course</th>
            <th>Semester</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student._id}>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.semester}</td>
              <td>{student.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```

In App.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
}
.container {
```

```
  width: 80%;
  margin: 40px auto;
  background-color: white;
  padding: 25px;
  border-radius: 8px;
}
h1 {
  text-align: center;
}
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}
th,
td {
  border: 1px solid #cccccc;
  padding: 12px;
  text-align: left;
}
th {
  background-color: #eeeeee;
}
```


File Edit Selection View Go Run Terminal Help
practical_8
Explorer
practical_8
Backend
student-frontend
student-frontend
http://localhost:5173/
Student Management System
Student Records Retrieved from Backend
Name
Course
Semester
City
Ansari Abid
B.Sc. Information Technology
5
Mumbai
Rahul Sharma
B.Sc. Information Technology
5
Mumbai
Khan Alqama
M.Sc. Information Technology
3
Thane
Ansari Salman
B.Sc. Information Technology
4
Kalyan
Riya Sharma
M.Sc. Information Technology
2
Pune
Problems
Output
Debug Console
Terminal
Ports
VITE v8.3.2 ready in 499 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

C. Add, Update and Delete Student Records.

In src/app.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";
function App() {
  const [students, setStudents] = useState([]);
  const [formData, setFormData] = useState({
  name: "",
    course: "",
    semester: "",
    city: "",
  });
  const [editingId, setEditingId] = useState(null);
  const [message, setMessage] = useState("");
  const apiUrl = "http://localhost:3000/students";
  const fetchStudents = async () => {
    try {
      const response = await axios.get(apiUrl);
      setStudents(response.data);
    } catch (error) {
      console.error("Error fetching students:", error);
    }
  };
  useEffect(() => {
    fetchStudents();
  }, []);
```

```
  const handleChange = (event) => {
    setFormData({ ...formData, [event.target.name]: event.target.value });
  };
  const handleSubmit = async (event) => {
    event.preventDefault();
    try {
      if (editingId) {
        await axios.put(`${apiUrl}/${editingId}`, formData);
        setEditingId(null);
        setMessage("Student updated successfully!");
      } else {
        await axios.post(apiUrl, formData);
        setMessage("Student added successfully!");
      }
      setFormData({ name: "", course: "", semester: "", city: "" });
      fetchStudents();
      setTimeout(() => setMessage(""), 3000);
    } catch (error) {
      console.error("Error saving student:", error);
      setMessage("Error saving student!");
    }
  };
  const handleEdit = (student) => {
    setFormData({
      name: student.name,
      course: student.course,
      semester: student.semester,
```

```
      city: student.city,
    });
    setEditingId(student._id);
  };
  const handleDelete = async (id) => {
    try {
      await axios.delete(`${apiUrl}/${id}`);
      fetchStudents();
      setMessage("Student deleted successfully!");
      setTimeout(() => setMessage(""), 3000);
    } catch (error) {
      console.error("Error deleting student:", error);
    }
  };
  return (
    <div className="container">
      <h1>Student Management System</h1>
      {message && (
        <p style={{ color: "green", textAlign: "center", fontWeight: "bold" }}>
          {message}
        </p>
      )}
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          name="name"
          placeholder="Enter student name"
```

```
          value={formData.name}
          onChange={handleChange}
          required
        />
        <input
          type="text"
          name="course"
          placeholder="Enter course"
          value={formData.course}
          onChange={handleChange}
          required
        />
        <input
          type="number"
          name="semester"
          placeholder="Enter semester"
          value={formData.semester}
          onChange={handleChange}
          required
        />
        <input
          type="text"
          name="city"
          placeholder="Enter city"
          value={formData.city}
          onChange={handleChange}
          required
```

```
        />
        <button type="submit">
          {editingId ? "Update Student" : "AddStudent"}
        </button>
      </form>
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Course</th>
            <th>Semester</th>
            <th>City</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student._id}>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.semester}</td>
              <td>{student.city}</td>
              <td>
                <button onClick={() => handleEdit(student)}>Edit</button>
                <button onClick={() => handleDelete(student._id)}>
                  Delete
                </button>
```

```
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 );
}
export default App;
```

In App.css

```
body {
 font-family: Arial, sans-serif;
 background-color: #f4f4f4;
 margin: 0;
}
.container {
 width: 85%;
 margin: 30px auto;
 background-color: white;
 padding: 25px;
 border-radius: 8px;
}
h1 {
 text-align: center;
}
form {
 display: flex;
```

```
  gap: 10px;
  flex-wrap: wrap;
  margin-bottom: 25px;
}
input {
  padding: 10px;
  flex: 1;
  min-width: 160px;
}
button {
  padding: 10px 15px;
  margin: 3px;
  cursor: pointer;
}
table {
  width: 100%;
  border-collapse: collapse;
}
th,
td {
  border: 1px solid #cccccc;
  padding: 10px;
  text-align: left;
}
th {
  background-color: #eeeeee;
}
```

Execution

Keep two terminals running.

Backend terminal

cd C:\MERN_Practicals\Practical_8

node server.js

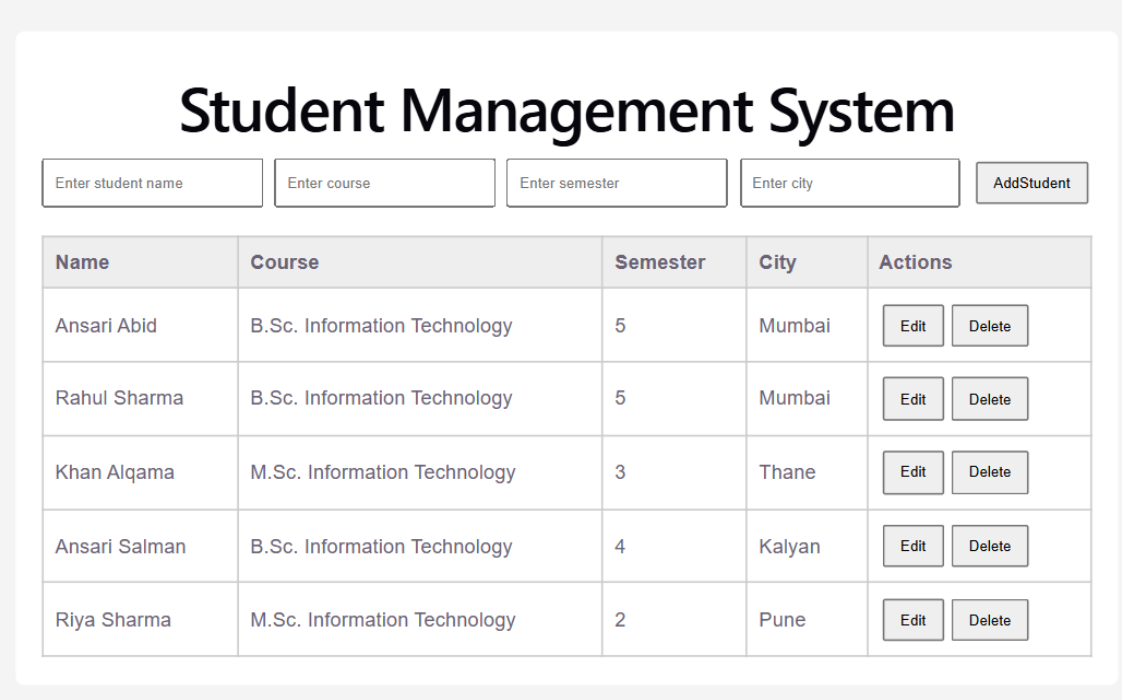
Frontend terminal

cd C:\MERN_Practicals\Practical_8\student-frontend

npm run dev

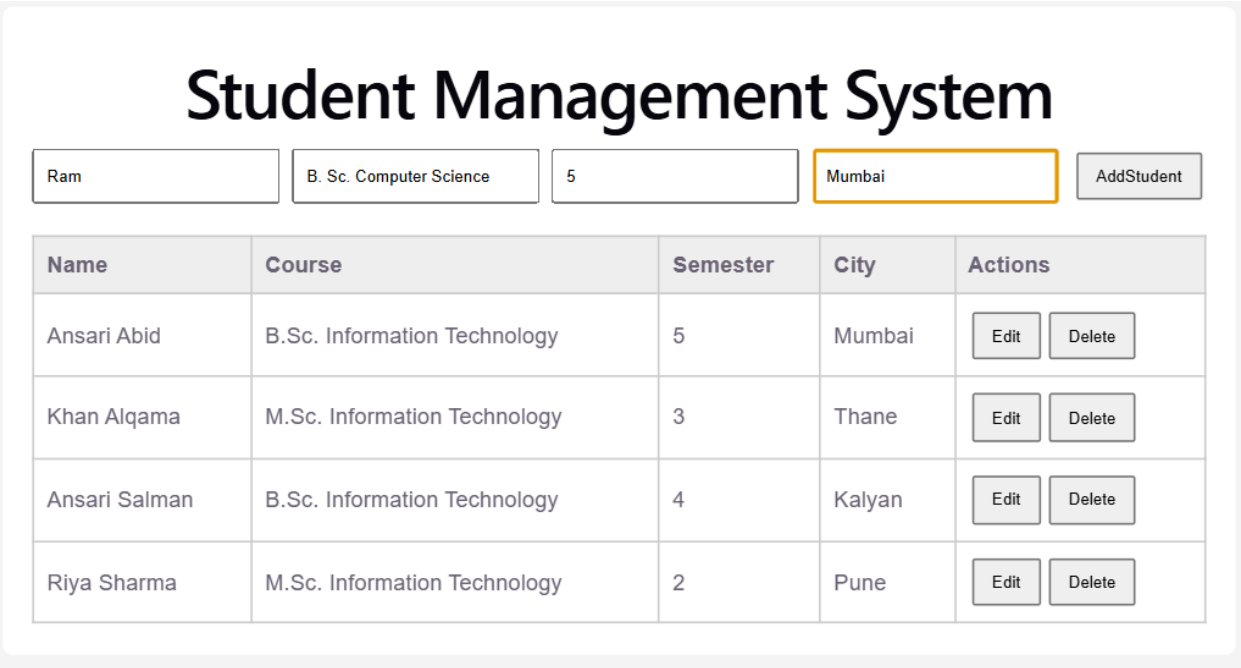
Open: http://localhost:5173

Output

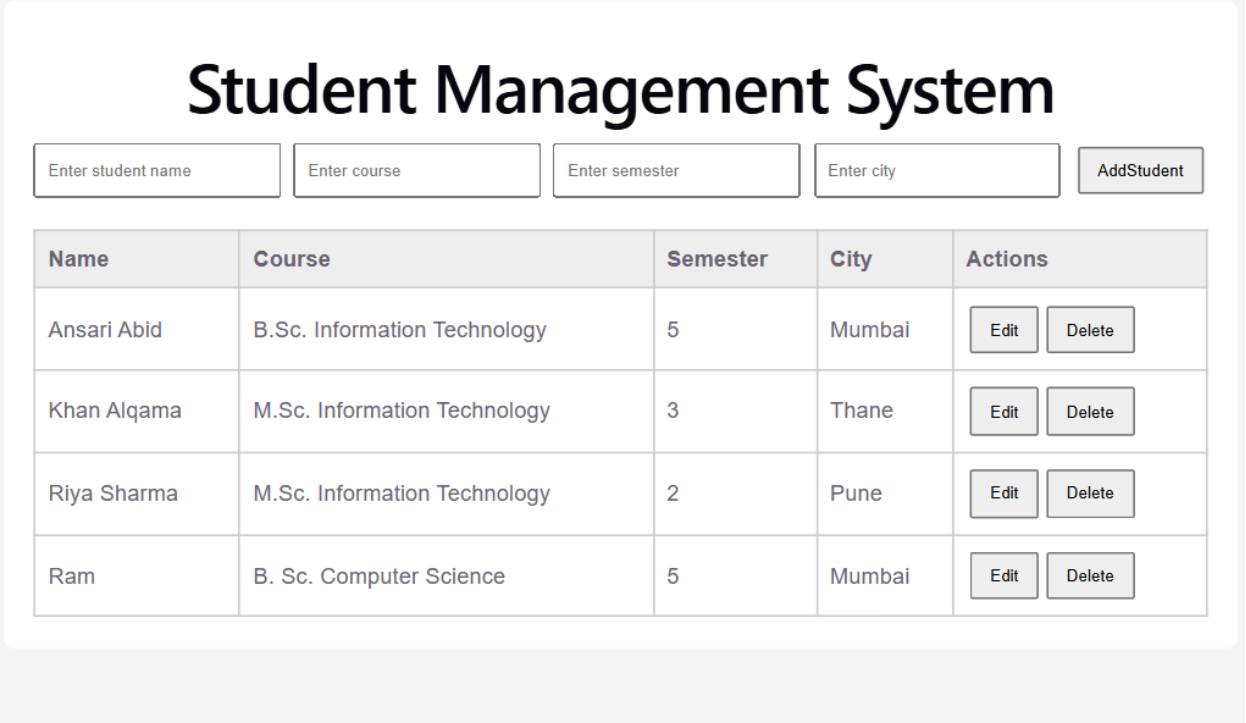


\

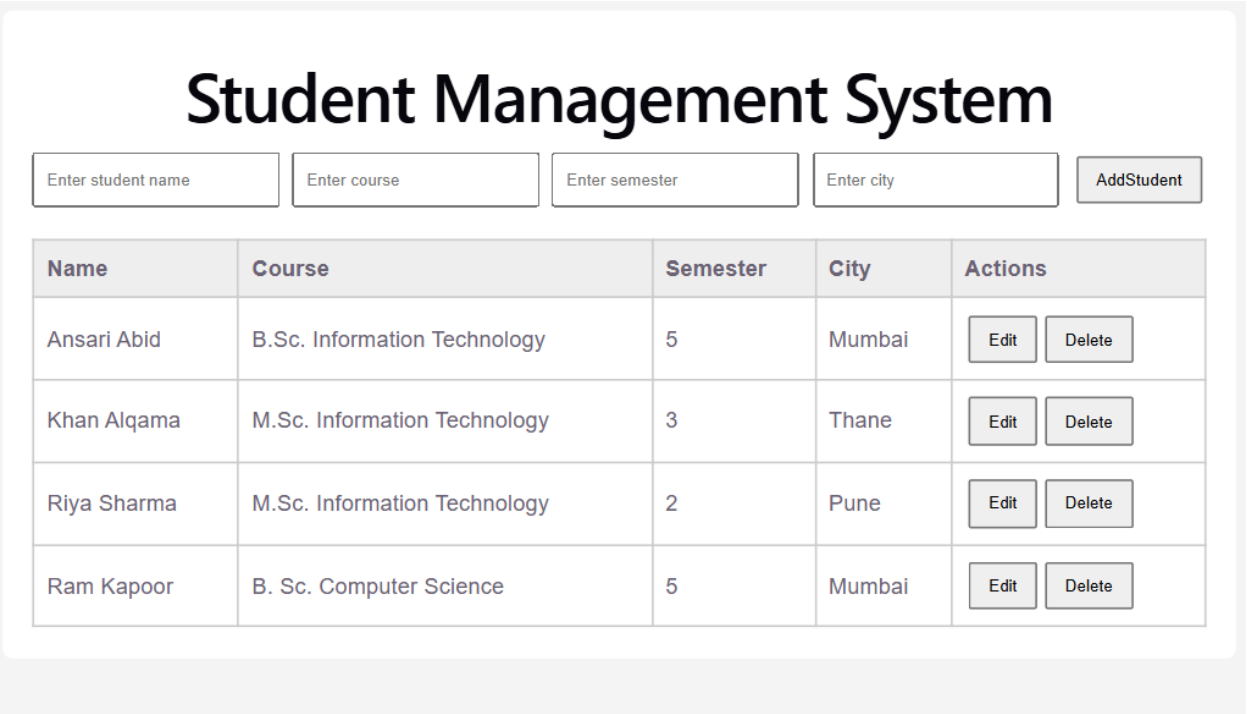
Add Student:



Delete Student:



Update Student:



Practical -10

Aim: Complete MERN CRUD Application.

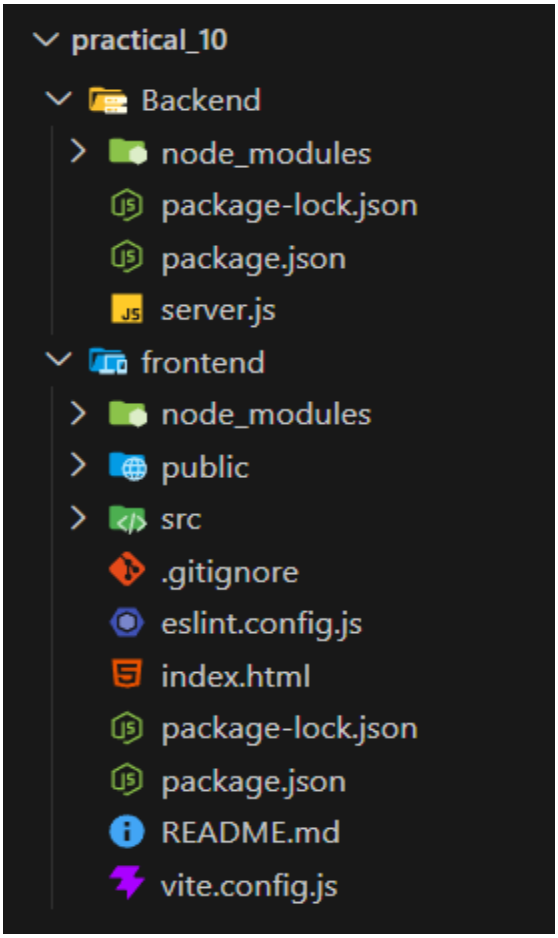
a. Design and Execute the Complete MERN Application.

To organize and execute the complete MERN application using separate frontend and backend folders.

Steps:

Create a folder named: Practical-10

Inside it maintain the following structure:



1. Create the Backend Folder

Open the backend folder in Visual Studio Code and initialize the Node.js project:

npm init –y

Install the required packages: npm install express mongoose cors

2: Create server.js

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 3000;
app.use(cors());
app.use(express.json());
mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log("MongoDB Connection Error:", error);
  });
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  course: {
    type: String,
    required: true,
  },
  semester: {
```

```
    type: Number,
    required: true,
  },
  city: {
    type: String,
    required: true,
  },
});
const Student = mongoose.model("Student", studentSchema);
// CREATE Student
app.post("/students", async (req, res) => {
  try {
    const student = new Student(req.body);
    const savedStudent = await student.save();
    res.status(201).json(savedStudent);
  } catch (error) {
    res.status(400).json({
      message: "Unable to add student",
      error: error.message,
    });
  }
});
// READ All Students
app.get("/students", async (req, res) => {
try {
    const students = await Student.find();
    res.json(students);
```

```
  } catch (error) {
    res.status(500).json({
      message: "Unable to retrieve students",
    });
  }
});
// UPDATE Student
app.put("/students/:id", async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,
        runValidators: true,
      },
    );
    res.json(updatedStudent);
  } catch (error) {
    res.status(400).json({
      message: "Unable to update student",
      error: error.message,
    });
  }
});
// DELETE Student
app.delete("/students/:id", async (req, res) => {
```

```
  try {
    await Student.findByIdAndDelete(req.params.id);
    res.json({
      message: "Student Deleted Successfully",
    });
  } catch (error) {
    res.status(400).json({
      message: "Unable to delete student",
    });
  }
});
app.listen(PORT, () => {
  console.log(`Server Running on Port ${PORT}`);
});
```

3. Create the React Frontend

Open another terminal inside the Practica-10 folder and execute:

npm create vite@latest frontend

Select:

Framework: React

Variant: JavaScript

When asked: Install with npm and start now?

Select: Yes

In App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";

const apiUrl = "http://localhost:3000/students";
```

```
async function request(url, options = {}) {
  const response = await fetch(url, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });

  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }

  return response.json();
}

function App() {
  const [students, setStudents] = useState([]);
  const [formData, setFormData] = useState({
  name: "",
    course: "",
    semester: "",
    city: "",
  });
  const [editingId, setEditingId] = useState(null);

  const fetchStudents = async () => {
    const data = await request(apiUrl);
    setStudents(data);
```

```
  };

  useEffect(() => {
    const loadStudents = async () => {
      try {
        await fetchStudents();
      } catch (error) {
        console.error("Unable to load students:", error);
      }
    };

    loadStudents();
  }, []);

  const handleChange = (event) => {
    setFormData({ ...formData, [event.target.name]: event.target.value });
  };

  const handleSubmit = async (event) => {
    event.preventDefault();

    try {
      if (editingId) {
        await request(`${apiUrl}/${editingId}`, {
          method: "PUT",
          body: JSON.stringify(formData),
        });
```

```
        setEditingId(null);
      } else {
        await request(apiUrl, {
          method: "POST",
          body: JSON.stringify(formData),
        });
      }

      setFormData({ name: "", course: "", semester: "", city: "" });
      await fetchStudents();
    } catch (error) {
      console.error("Unable to save student:", error);
    }
  };

  const handleEdit = (student) => {
    setFormData({
      name: student.name,
      course: student.course,
      semester: student.semester,
      city: student.city,
    });
    setEditingId(student._id);
  };

  const handleDelete = async (id) => {
    try {
```

```
      await request(`${apiUrl}/${id}`, { method: "DELETE" });
      await fetchStudents();
    } catch (error) {
      console.error("Unable to delete student:", error);
    }
  };
  return (
    <div className="main-bg">
      <header className="top-header">
        MERN Student Management System
        <span>React + Express + MongoDB</span>
      </header>
      <div className="dashboard">
        <h2>Complete MERN Application Dashboard</h2>
        <div className="stats">
          <div className="stat-box">
            <p>Total Students</p>
            <h3>{students.length}</h3>
          </div>
          <div className="stat-box">
            <p>Active Semester</p>
            <h3>5</h3>
          </div>
          <div className="stat-box">
            <p>Database Status</p>
            <h3 className="connected">Connected</h3>
          </div>
```

```
      </div>
      <form onSubmit={handleSubmit} className="dash-form">
        <input
          type="text"
          name="name"
          placeholder="Enter student name"
          value={formData.name}
          onChange={handleChange}
          required
        />
        <input
          type="text"
          name="course"
          placeholder="Enter course"
          value={formData.course}
          onChange={handleChange}
          required
        />
        <input
          type="number"
          name="semester"
          placeholder="Enter semester"
          value={formData.semester}
          onChange={handleChange}
          required
        />
        <input
```

```
          type="text"
          name="city"
          placeholder="Enter city"
          value={formData.city}
          onChange={handleChange}
          required
        />
        <button type="submit" className="add-btn">
          {editingId ? "Update" : "Add Student"}
        </button>
      </form>
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Course</th>
            <th>Semester</th>
            <th>City</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student._id}>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.semester}</td>
```

```
            <td>{student.city}</td>
            <td>
              <button
                className="edit-btn"
                onClick={() => handleEdit(student)}
              >
                Edit
              </button>
              <button
                className="del-btn"
                onClick={() => handleDelete(student._id)}
              >
                Delete
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
    <p className="footer">
      B.Sc. Information Technology - Full Stack Development MERN Practice
    </p>
  </div>
 </div>
 );
}
export default App;
```

In App.css

```
body {
margin: 0;
  font-family: Arial, sans-serif;
  background: #eef1f5;
}
.main-bg {
  min-height: 100vh;
}
.top-header {
  background: #1a1a2e;
  color: white;
  padding: 12px 25px;
  font-weight: bold;
  display: flex;
  justify-content: space-between;
}
.top-header span {
  font-size: 12px;
  color: #aaa;
}
.dashboard {
  width: 90%;
  max-width: 1000px;
  background: white;
  margin: 20px auto;
  padding: 25px;
```

```
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
.dashboard h2 {
  margin-top: 0;
}
.stats {
  display: flex;
  gap: 15px;
  margin: 20px 0;
}
.stat-box {
  flex: 1;
  background: #f8f9fa;
  padding: 15px;
  border-radius: 8px;
  border: 1px solid #eee;
}
.stat-box p {
  margin: 0;
  font-size: 12px;
  color: #666;
}
.stat-box h3 {
  margin: 5px 0 0 0;
  font-size: 20px;
}
```

```
.connected {
  color: #1a1a2e;
}
.dash-form {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-bottom: 20px;
}
.dash-form input {
  padding: 8px 10px;
  flex: 1;
  min-width: 130px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.add-btn {
  background: #1a1a2e;
  color: white;
  border: none;
  padding: 8px 15px;
  border-radius: 5px;
  cursor: pointer;
}
table {
  width: 100%;
  border-collapse: collapse;
```

```
}
th {
  background: #f1f1f1;
  text-align: left;
  padding: 10px;
  font-size: 13px;
}
td {
  padding: 10px;
  border-bottom: 1px solid #eee;
  font-size: 13px;
}
.edit-btn {
  background: #6c757d;
  color: white;
  border: none;
  padding: 5px 10px;
  border-radius: 4px;
  margin-right: 5px;
  cursor: pointer;
}
.del-btn {
  background: #1a1a2e;
  color: white;
  border: none;
  padding: 5px 10px;
  border-radius: 4px;
```

```
  cursor: pointer;
}
.footer {
  text-align: center;
  font-size: 11px;
  color: #888;
  margin-top: 20px;
}
```

If the application does not start automatically, execute:

cd frontend

npm install

npm run dev

4. Install Axios

Inside the frontend folder, execute: npm install axios

5. Run the Backend

Open a terminal inside the backend folder:

node server.js

Expected terminal output:

```
PS D:\MERN_Practicals\practical_10\Backend> node server.js
Server Running on Port 3000
MongoDB Connected Successfully
```

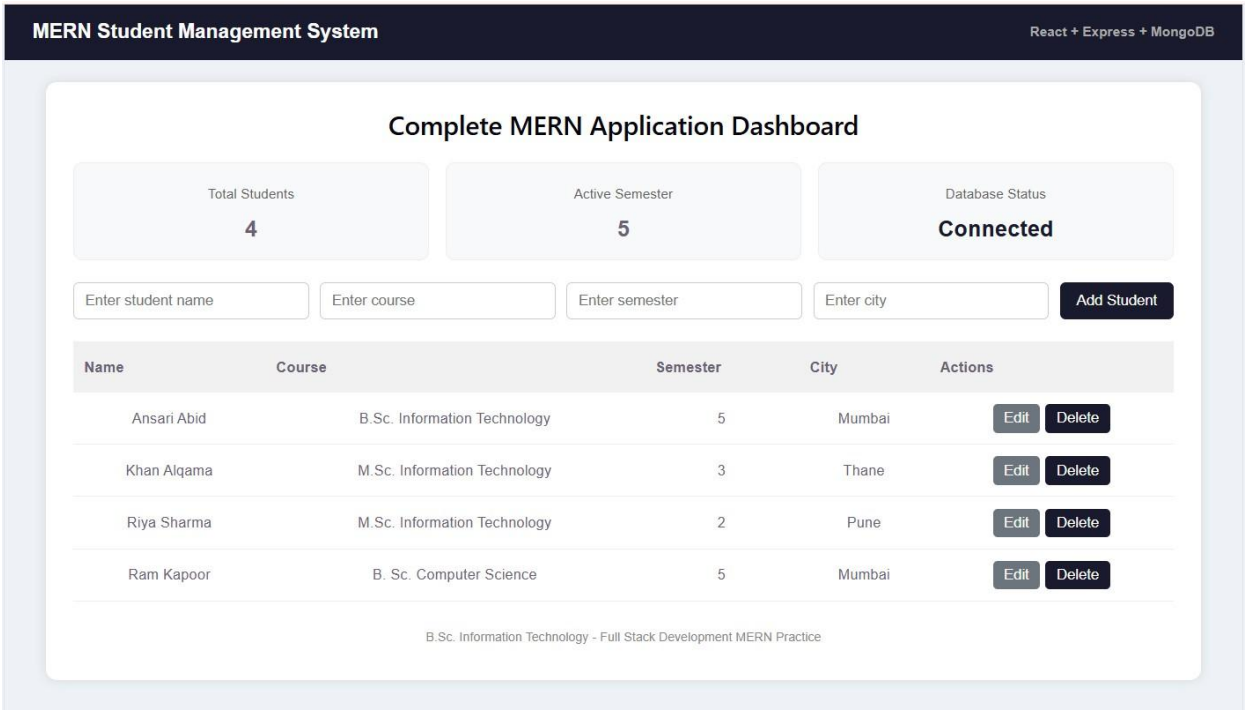
6: Run the Frontend

Open another terminal inside the frontend folder:

npm run dev

Open the displayed URL: http://localhost:5173/

Output:


MERN Student Management System
React + Express + MongoDB
Complete MERN Application Dashboard
Total Students
4
Active Semester
5
Database Status
Connected
Enter student name
Enter course
Enter semester
Enter city
Add Student
Name
Course
Semester
City
Actions
Ansari Abid
B.Sc. Information Technology
5
Mumbai
Edit
Delete
Khan Alqama
M.Sc. Information Technology
3
Thane
Edit
Delete
Riya Sharma
M.Sc. Information Technology
2
Pune
Edit
Delete
Ram Kapoor
B. Sc. Computer Science
5
Mumbai
Edit
Delete
B.Sc. Information Technology - Full Stack Development MERN Practice